Prédiction du délai de retour à l'emploi - notebook principal

Ce notebook présente la démarche complète du projet : cadrage métier, préparation des données, modélisation multiclasse, analyse des risques de biais et trajectoire d'industrialisation.

Sommaire interactif

Cliquez sur une section ou un sous-titre pour accéder directement au passage correspondant.

1. Cadrage métier et problème
   - Contexte client
   - Énoncé du problème IA
   - Critères de succès
   - Risques éthiques et réglementaires
2. Identification et acquisition des données
   - Sources
   - Dictionnaire de variables
3. Exploration et analyse des données
   - Qualité des données
   - Distributions des variables
   - Analyse des biais et variables sensibles
4. Préparation des données
   - Train/Test Split stratifié
   - Pipeline de préprocessing
   - Scénarios de jeux de données
5. Modélisation et entraînement
   - Modèles candidats
   - Entraînement et validation croisée
   - Comparaison et benchmark
   - Optimisation des hyperparamètres
   - Évaluation finale
   - Synthèse modélisation
6. Analyse des scénarios et arbitrages chiffrés
   - Tableau comparatif chiffré
   - Recommandation finale
   - Synthèse des arbitrages
7. Interprétation et communication métier
   - Variables explicatives
   - Analyse des erreurs et fallback
   - Calibration OOF du seuil

    - Message au client
8. Industrialisation
    - Architecture SI
    - Contrat API
    - Modèle, artefacts et retrain
    - Observabilité
    - CI/CD
    - Dossier de livraison et preuves
9. Suivi en production et amélioration continue
    - Métriques à surveiller
    - Boucle de rétroaction
    - Plan de réentraînement
    - Robustesse en exploitation
10. Annexes

- Glossaire
- Sources et documentation
- Traçabilité des décisions
- Journal de bord complet
- Journal de bord synthétique
- Vérification finale

Retour au début

Fil directeur de la soutenance : partir du besoin métier, mesurer le prix de la décision éthique, calibrer le garde-fou hors test, puis démontrer que le modèle est livré, surveillé et réversible.

0. Imports et configuration

Cette section centralise les imports, les constantes de reproductibilité et les chemins projet. Le but est d'avoir un point d'entrée unique pour exécuter le notebook de manière stable.

In [1]:
```python
# Imports standards
from pathlib import Path
import warnings

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Reproductibilité
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

# Affichage
pd.set_option("display.max_columns", 50)
pd.set_option("display.width", 140)
```

```
sns.set_theme(style="whitegrid")
warnings.filterwarnings("ignore")

# Chemins du projet (robuste quel que soit le dossier de lancement)
PROJECT_ROOT = Path.cwd()
if PROJECT_ROOT.name == "notebooks":
    PROJECT_ROOT = PROJECT_ROOT.parent

DATA_DIR = PROJECT_ROOT / "data"
DATA_RAW_DIR = DATA_DIR / "raw"
REPORTS_DIR = PROJECT_ROOT / "reports"
MODELS_DIR = PROJECT_ROOT / "models"
```

0.5 Traçabilité de la session

Cette section enregistre les éléments nécessaires à la reproductibilité :

- version de Python et bibliothèques clés ;
- commit Git ;
- source, version et empreinte du dataset ;
- date/heure d'exécution.

Ces informations facilitent l'audit des résultats et le suivi des évolutions du projet.

In [2]:
```
import hashlib
import subprocess
import sys
from datetime import datetime
from importlib.metadata import version, PackageNotFoundError

# Localisation robuste du dataset (priorité à data/raw, fallback data/)
dataset_candidates = sorted(DATA_RAW_DIR.glob("*.csv"))
if not dataset_candidates:
    dataset_candidates = sorted(DATA_DIR.glob("*.csv"))

if not dataset_candidates:
    raise FileNotFoundError("Aucun fichier CSV trouvé dans data/raw ou data.")

DATASET_PATH = dataset_candidates[0]
DATASET_NAME = DATASET_PATH.name
try:
    DATASET_SOURCE = DATASET_PATH.relative_to(PROJECT_ROOT).as_posix()
except ValueError:
    # Fallback défensif : ne jamais exposer de chemin absolu
    DATASET_SOURCE = DATASET_NAME

DATASET_VERSION = datetime.fromtimestamp(DATASET_PATH.stat().st_mtime).strftime(

# Empreinte du dataset pour la traçabilité
with open(DATASET_PATH, "rb") as f:
    dataset_sha256 = hashlib.sha256(f.read()).hexdigest()

# Métadonnées de session
session_date = datetime.now().isoformat(timespec="minutes")
py_version = sys.version.split()[0]

try:
```

```
    git_commit = subprocess.check_output(
        ["git", "rev-parse", "--short", "HEAD"], stderr=subprocess.DEVNULL
    ).decode().strip()
except (subprocess.CalledProcessError, FileNotFoundError):
    git_commit = "non disponible (notebook hors repo Git)"

packages_to_track = ["numpy", "pandas", "scikit-learn", "xgboost", "lightgbm", "
tracked_versions = {}
for pkg in packages_to_track:
    try:
        tracked_versions[pkg] = version(pkg)
    except PackageNotFoundError:
        tracked_versions[pkg] = "non installé"

print(f"Date session : {session_date}")
print(f"Python       : {py_version}")
print(f"Git commit   : {git_commit}")
print(f"Dataset      : {DATASET_NAME}")
print(f"Source       : {DATASET_SOURCE}")
print(f"Version data : {DATASET_VERSION}")
print(f"SHA256       : {dataset_sha256}")
print("\nVersions bibliothèques clés :")
for pkg, ver in tracked_versions.items():
    print(f"- {pkg:12s}: {ver}")
```

```
Date session : 2026-10-04T23:03
Python       : 3.12.10
Git commit   : 3236014
Dataset      : dataset_trajectoire_emploi_Sujet Examen CISIA - Promo Upskilling A
tlas - mai-oct2026 (Session-00279143).csv
Source       : data/raw/dataset_trajectoire_emploi_Sujet Examen CISIA - Promo Ups
killing Atlas - mai-oct2026 (Session-00279143).csv
Version data : 2026-07-16 09:20
SHA256       : 2b9cb8c813b5f97540f0130b5e1cae2bad7ce5cbc1f57ffce51f54b498a7b34e

Versions bibliothèques clés :
- numpy       : 2.5.1
- pandas      : 2.3.3
- scikit-learn: 1.9.0
- xgboost     : 3.3.0
- lightgbm    : 4.6.0
- mlflow      : 3.14.0
```

---

1. Cadrage métier et problème

Cette section formalise le besoin métier, les critères de succès et les contraintes éthiques et réglementaires avant la phase de modélisation.

1.1 Contexte client

Le client est une agence nationale de l'emploi qui souhaite améliorer l'orientation des demandeurs d'emploi dès le premier entretien. L'enjeu opérationnel est de classer chaque situation dans un niveau de risque de retour à l'emploi :

- **Classe 0** : retour rapide (< 6 mois) ;

- **Classe 1** : retour moyen (6 à 12 mois) ;
- **Classe 2** : risque de chômage de longue durée (> 12 mois).

L'objectif n'est pas d'automatiser la décision, mais de fournir aux conseillers un outil d'aide à l'orientation homogène, traçable et réactif. Cette aide doit respecter les principes fondamentaux du service public : explicabilité, confidentialité, robustesse et stricte non-discrimination.

1.2 Énoncé du problème IA

Tâche IA retenue : **classification supervisée multiclasse (3 classes)**.

Formulation : prédire `classe_retour_emploi` à partir de :

- variables tabulaires (âge, diplôme, ancienneté, allocation, géographie),
- variable textuelle `synthese_entretien` (synthèse rédigée par le conseiller),
- avec comparaison de 4 scénarios de données (multimodal complet, sans variable sensible, texte seul, tabulaire seul).

1.3 Bénéficiaires et impacts

Bénéficiaires directs :

- conseillers emploi lors des entretiens d'orientation,
- managers opérationnels pilotant la répartition des moyens d'accompagnement.

Bénéficiaires indirects :

- demandeurs d'emploi, dont l'accès à un accompagnement renforcé dépend de la qualité de l'orientation initiale,
- administration publique (efficacité des politiques d'emploi, sobriété des ressources).

Impact critique métier :

- l'erreur la plus préjudiciable est la sous-estimation du risque : une personne relevant réellement de la **classe 2 prédite à tort en classe 0** entraîne une perte de chance d'accompagnement renforcé précoce.

1.4 Critères de succès

Synthèse des objectifs initiaux et des cibles révisées après l'analyse exploratoire (EDA) :

Type	Critère	Cible initiale (client)	Cible révisée après EDA	Justification de la révision
Métier	Réduire les erreurs critiques (2 -> 0)	<= 5 % du test set	<= 2 % résiduel après escalade	Sécurisation renforcée via une politique d'abstention humaine (`A_REVOIR`).
Modèle	F1-score macro	>= 0.70	>= 0.70 (validé ~0,71	Maintien d'un équilibre global de prédiction sur

Type	Critère	Cible initiale (client)	Cible révisée après EDA	Justification de la révision
			sur S2)	les 3 classes.
Modèle	Recall classe 2	>= 0.80	0,55 à 0,60 brut (complété par escalade)	Déséquilibre de la classe 2 (18,1 %) nécessitant un seuil de rejet pour atteindre la couverture visée.
Opérationnel	Latence inférence API	< 300 ms en p95	< 300 ms en p95	Compatibilité confirmée avec un usage fluide pendant l'entretien.
Gouvernance	Traçabilité décisionnelle	100 % des inférences journalisées	100 % journalisé	Traçabilité complète via logs JSONL, Prometheus et MLflow.

1.5 Risques éthiques et réglementaires anticipés

Risques majeurs identifiés :

- **Discrimination directe** : utilisation de la variable `nationalite_hors_ue` (traitée par exclusion stricte dans le scénario de production S2).
- **Discrimination indirecte via proxys** : l'âge, le statut allocataire et le département peuvent réintroduire des biais indirects. L'âge est maintenu pour des raisons d'accompagnement métier, mais suivi par tranche d'âge.
- **Biais de formulation textuelle** : les verbatims conseillers peuvent véhiculer des stéréotypes ou des styles rédactionnels hétérogènes.

Cadre réglementaire :

- RGPD : minimisation des données, finalité explicite, sécurité et journalisation proportionnée.
- Principes de non-discrimination applicables au service public.
- Exigence d'explicabilité suffisante pour motiver la recommandation d'orientation.

1.6 Synthèse cadrage

Le cas d'usage est un problème de classification multiclasse à fort enjeu social. La stratégie de modélisation privilégie la conformité et la sécurité métier : le scénario de production S2 exclut la nationalité, utilise un encodage ordinal pour le diplôme et combine la prédiction probabiliste avec une règle d'escalade humaine pour les cas ambigus.

---

2. Identification et acquisition des données

Cette section documente la source, le format, la volumétrie et la traçabilité du jeu de données avant tout traitement. L'objectif est de valider l'adéquation entre la donnée

disponible et le besoin métier.

2.1 Sources

Source	Type	Volumétrie	Format	Accès	Date extraction
Jeu fourni dans le sujet d'examen CISIA (fichier local)	Table tabulaire enrichie d'un champ texte	2 500 lignes, 10 colonnes	CSV	Lecture locale (repo, dossier `data/raw`)	16/07/2026

Localisation projet : `data/raw/dataset_trajectoire_emploi_Sujet Examen CISIA - Promo Upskilling Atlas - mai-oct2026 (Session-00279143).csv`

Traçabilité : version source, SHA256 et contexte d'exécution sont suivis en section 0.5.

In [3]:
```python
# 2.2 Chargement
from pathlib import Path

import pandas as pd

# On privilégie la variable DATASET_PATH définie en section 0.
# Fallback : on cherche le premier CSV dans data/raw.
if "DATASET_PATH" in globals() and Path(DATASET_PATH).exists():
    dataset_path = Path(DATASET_PATH)
else:
    raw_dir = Path("data/raw")
    candidates = sorted(raw_dir.glob("*.csv"))
    if not candidates:
        raise FileNotFoundError("Aucun CSV trouvé dans data/raw")
    dataset_path = candidates[0]

df = pd.read_csv(dataset_path)

# Affichage sans chemin absolu
project_root = Path(PROJECT_ROOT) if "PROJECT_ROOT" in globals() else Path.cwd()
dataset_rel = dataset_path.resolve().relative_to(project_root.resolve())

print(f"Dataset chargé : {dataset_rel.as_posix()}")
print(f"Dimensions     : {df.shape[0]} lignes x {df.shape[1]} colonnes")
print("\nColonnes :")
print(df.columns.tolist())

display(df.head(3))
```

```
Dataset chargé : data/raw/dataset_trajectoire_emploi_Sujet Examen CISIA - Promo U
pskilling Atlas - mai-oct2026 (Session-00279143).csv
Dimensions     : 2500 lignes x 10 colonnes

Colonnes :
['usager_id', 'age', 'niveau_diplome', 'anciennete_poste_ans', 'code_rome_vise',
'code_insee_commune', 'est_allocataire', 'nationalite_hors_ue', 'synthese_entreti
en', 'classe_retour_emploi']
```

	usager_id	age	niveau_diplome	anciennete_poste_ans	code_rome_vise	code_insee_cor
0	ID_0000	56.0	Bac+2	2.6	D1503	
1	ID_0001	46.0	Sans diplôme	10.5	G1302	
2	ID_0002	NaN	Bac	1.4	M1705	

2.3 Dictionnaire de variables

Variable	Description	Type	Plage / valeurs	Cible ?	Sensible ?
`usager_id`	Identifiant technique du dossier usager	catégorielle (ID)	`ID_0000` … `ID_2499`	❌	❌
`age`	Âge de l'usager	numérique	valeurs positives (années), avec manquants observés	❌	✅ (proxy possible)
`niveau_diplome`	Niveau de diplôme déclaré	catégorielle ordonnée	ex. `Sans diplôme` , `Bac` , `Bac+2` , …	❌	❌
`anciennete_poste_ans`	Ancienneté sur le dernier poste (années)	numérique	valeurs réelles >= 0	❌	❌
`code_rome_vise`	Code métier visé (ROME)	catégorielle	codes alphanumériques (ex. `D1503`)	❌	❌
`code_insee_commune`	Code INSEE de la commune	catégorielle	codes commune (5 caractères)	❌	✅ (proxy territorial)
`est_allocataire`	Statut allocataire (oui/non)	binaire	`0` / `1` + manquants	❌	✅ (proxy socio-éco)
`nationalite_hors_ue`	Indicateur de nationalité hors UE	binaire	`0` / `1`	❌	✅ (variable sensible)
`synthese_entretien`	Texte libre de synthèse	texte	verbatim court à moyen	❌	✅ (peut contenir

Variable	Description	Type	Plage / valeurs	Cible ?	Sensible ?
	conseiller				des signaux sensibles)
`classe_retour_emploi`	Classe cible de délai de retour à l'emploi	catégorielle (3 classes)	`0` , `1` , `2`	✅	❌

2.4 Synthèse acquisition

Le dataset est cohérent avec le besoin métier posé en section 1 : il combine des variables socio-professionnelles tabulaires et un texte d'entretien, ce qui permet une approche multimodale.

Points d'attention détectés dès l'acquisition :

- présence de variables sensibles ou de proxys potentiels (`nationalite_hors_ue` , `age` , `code_insee_commune` , `est_allocataire`),
- présence de valeurs manquantes visibles dès l'aperçu sur certaines colonnes (`age` , `est_allocataire`),
- présence d'un champ texte non structuré (`synthese_entretien`) nécessitant une préparation dédiée.

Ces constats seront quantifiés et documentés dans l'EDA (section 3) avant le design final des pipelines de préparation (section 4).

---

3. Exploration et analyse des données (EDA)

L'EDA vise à qualifier la donnée avant modélisation : qualité, distributions, déséquilibres et signaux de biais. Chaque analyse est reliée à une décision de préparation ou de modélisation.

3.1 Premier coup d'œil

Vérification de la structure générale du dataset : dimensions, aperçu, types et statistiques descriptives.

In [4]:
```
# Pré-requis section 3
assert "df" in globals(), "Le DataFrame df doit être chargé en section 2.2"

display(df.head(5))
print(f"Dimensions : {df.shape[0]} lignes x {df.shape[1]} colonnes")

print("\nTypes de variables :")
display(df.dtypes.to_frame("dtype"))
```

```
print("\nDescription statistique (numériques) :")
display(df.describe(include=["number"]).T)

print("\nDescription statistique (catégorielles) :")
display(df.describe(include=["object", "category"]).T)
```

	usager_id	age	niveau_diplome	anciennete_poste_ans	code_rome_vise	code_insee_cor
0	ID_0000	56.0	Bac+2	2.6	D1503	
1	ID_0001	46.0	Sans diplôme	10.5	G1302	
2	ID_0002	NaN	Bac	1.4	M1705	
3	ID_0003	60.0	Bac	0.2	A1203	
4	ID_0004	25.0	Sans diplôme	1.2	N1301	

```
Dimensions : 2500 lignes x 10 colonnes

Types de variables :
```

	dtype
usager_id	object
age	float64
niveau_diplome	object
anciennete_poste_ans	float64
code_rome_vise	object
code_insee_commune	object
est_allocataire	float64
nationalite_hors_ue	int64
synthese_entretien	object
classe_retour_emploi	int64

```
Description statistique (numériques) :
```

	count	mean	std	min	25%	50%	75%	max
age	2378.0	40.566442	13.225242	18.0	29.0	41.0	52.0	63.0
anciennete_poste_ans	2500.0	2.971600	2.979548	0.0	0.8	2.0	4.1	22.6
est_allocataire	2456.0	0.595684	0.490859	0.0	0.0	1.0	1.0	1.0
nationalite_hors_ue	2500.0	0.118000	0.322673	0.0	0.0	0.0	0.0	1.0
classe_retour_emploi	2500.0	0.806400	0.719671	0.0	0.0	1.0	1.0	2.0

Description statistique (catégorielles) :

	count	unique	top	freq
usager_id	2500	2500	ID_0000	1
niveau_diplome	2416	4	Bac	966
code_rome_vise	2500	50	H1206	71
code_insee_commune	2500	2407	46283	3
synthese_entretien	2419	9	Compétences à réactualiser sur les outils numé...	338

3.2 Qualité des données

Contrôle des valeurs manquantes, des doublons et détection simple d'aberrations sur variables numériques (règle IQR).

In [5]:
```
import matplotlib.pyplot as plt

n_rows = len(df)

missing = (
    df.isna().sum()
    .to_frame("nb_manquants")
    .assign(pct_manquants=lambda x: (x["nb_manquants"] / n_rows * 100).round(2))
    .sort_values("pct_manquants", ascending=False)
)
print("Valeurs manquantes par variable :")
display(missing)

# Graphique 1 : taux de manquants par variable
fig, ax = plt.subplots(figsize=(9, 4))
missing["pct_manquants"].plot(kind="bar", ax=ax, color="#e76f51")
ax.set_title("Taux de valeurs manquantes par variable")
ax.set_ylabel("% manquants")
ax.set_xlabel("")
ax.tick_params(axis="x", rotation=45)
plt.tight_layout()
plt.show()

nb_duplicates = int(df.duplicated().sum())
print(f"Doublons stricts (lignes complètes) : {nb_duplicates}")

num_cols = df.select_dtypes(include=["number"]).columns.tolist()
outliers = []
```

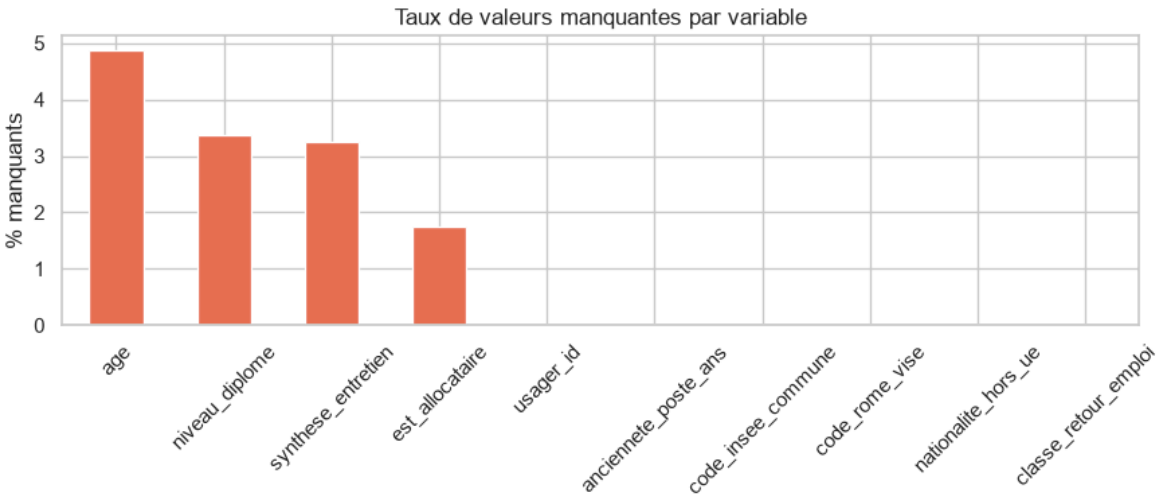
```python
for col in num_cols:
    s = df[col].dropna()
    if s.empty:
        continue
    q1, q3 = s.quantile(0.25), s.quantile(0.75)
    iqr = q3 - q1
    low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    n_out = int(((s < low) | (s > high)).sum())
    outliers.append({"variable": col, "nb_outliers_iqr": n_out, "pct_outliers_iq

outliers_df = pd.DataFrame(outliers).sort_values("pct_outliers_iqr", ascending=F
print("\nDétection simple d'aberrations (règle IQR) :")
display(outliers_df)

# Graphique 2 : part d'observations hors bornes IQR
fig, ax = plt.subplots(figsize=(8, 4))
outliers_df.plot(kind="bar", x="variable", y="pct_outliers_iqr", ax=ax, legend=F
ax.set_title("Part d'outliers (règle IQR) par variable")
ax.set_ylabel("% outliers")
ax.set_xlabel("")
ax.tick_params(axis="x", rotation=45)
plt.tight_layout()
plt.show()
```

```
Valeurs manquantes par variable :
```

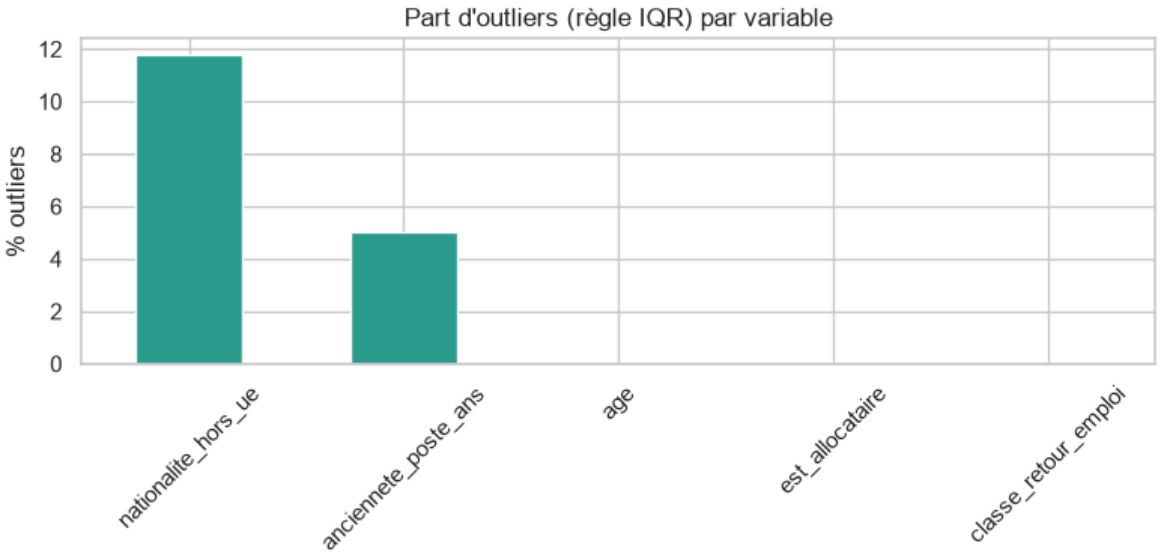
	nb_manquants	pct_manquants
age	122	4.88
niveau_diplome	84	3.36
synthese_entretien	81	3.24
est_allocataire	44	1.76
usager_id	0	0.00
anciennete_poste_ans	0	0.00
code_insee_commune	0	0.00
code_rome_vise	0	0.00
nationalite_hors_ue	0	0.00
classe_retour_emploi	0	0.00



```
Doublons stricts (lignes complètes) : 0

Détection simple d'aberrations (règle IQR) :
```

	variable	nb_outliers_iqr	pct_outliers_iqr
3	nationalite_hors_ue	295	11.80
1	anciennete_poste_ans	126	5.04
0	age	0	0.00
2	est_allocataire	0	0.00
4	classe_retour_emploi	0	0.00



3.3 Distributions des variables

Exploration des distributions numériques et des modalités catégorielles les plus fréquentes.

In [6]:
```
import matplotlib.pyplot as plt

num_cols = df.select_dtypes(include=["number"]).columns.tolist()
cat_cols = df.select_dtypes(include=["object", "category"]).columns.tolist()

print("Variables numériques :")
display(df[num_cols].describe().T)

# Graphique 1 : distributions de variables numériques clés
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
df["age"].dropna().plot(kind="hist", bins=20, ax=axes[0], color="#264653")
axes[0].set_title("Distribution de l'âge")
axes[0].set_xlabel("âge")

df["anciennete_poste_ans"].dropna().plot(kind="hist", bins=20, ax=axes[1], color
axes[1].set_title("Distribution de l'ancienneté")
axes[1].set_xlabel("années")
plt.tight_layout()
plt.show()

print("\nSynthèse des variables catégorielles (cardinalité + manquants) :")
```

```python
cat_summary = pd.DataFrame({
    "variable": cat_cols,
    "nb_modalites": [df[c].nunique(dropna=True) for c in cat_cols],
    "pct_manquants": [round(df[c].isna().mean() * 100, 2) for c in cat_cols],
}).sort_values("nb_modalites", ascending=False)
display(cat_summary)

# Affichage compact par défaut
if "niveau_diplome" in df.columns:
    diplome_counts = df["niveau_diplome"].fillna("<NA>").value_counts()
    fig, ax = plt.subplots(figsize=(7, 4))
    diplome_counts.plot(kind="bar", ax=ax, color="#457b9d")
    ax.set_title("Répartition du niveau de diplôme")
    ax.set_ylabel("effectif")
    ax.set_xlabel("")
    ax.tick_params(axis="x", rotation=25)
    plt.tight_layout()
    plt.show()

# Optionnel : passer à True pour le détail des top modalités
SHOW_DETAILED_CATS = False
if SHOW_DETAILED_CATS:
    focus_cats = ["niveau_diplome", "code_rome_vise", "synthese_entretien"]
    print("\nTop modalités (aperçu ciblé) :")
    for col in focus_cats:
        if col not in df.columns:
            continue
        topk = df[col].fillna("<NA>").value_counts(dropna=False).head(5)
        topk_df = topk.rename_axis(col).to_frame("effectif")
        topk_df["part_pct"] = (topk_df["effectif"] / len(df) * 100).round(2)

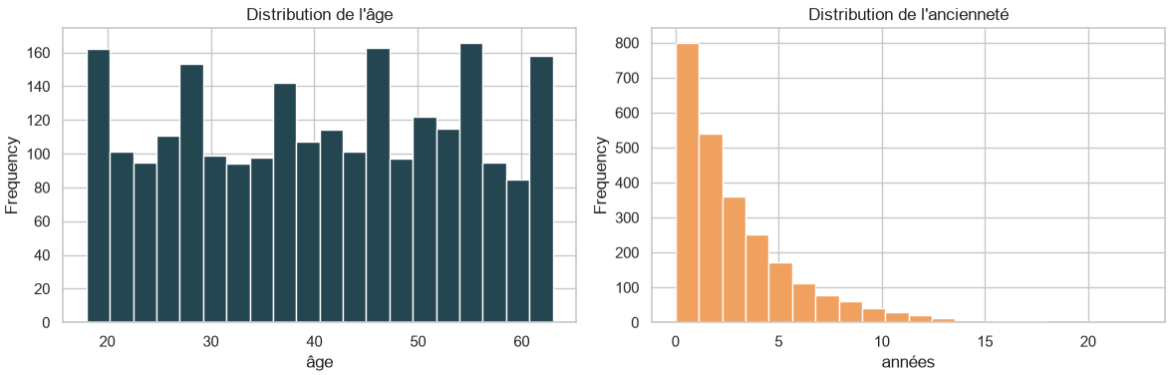
        topk_plot = topk.copy()
        topk_plot.index = [str(v)[:70] + ("..." if len(str(v)) > 70 else "") for

        print(f"\n- {col}")
        display(topk_df)

        fig, ax = plt.subplots(figsize=(8, 3.5))
        topk_plot.sort_values().plot(kind="barh", ax=ax, color="#457b9d")
        ax.set_title(f"Top 5 modalités - {col}")
        ax.set_xlabel("effectif")
        ax.set_ylabel("")
        plt.tight_layout()
        plt.show()
```

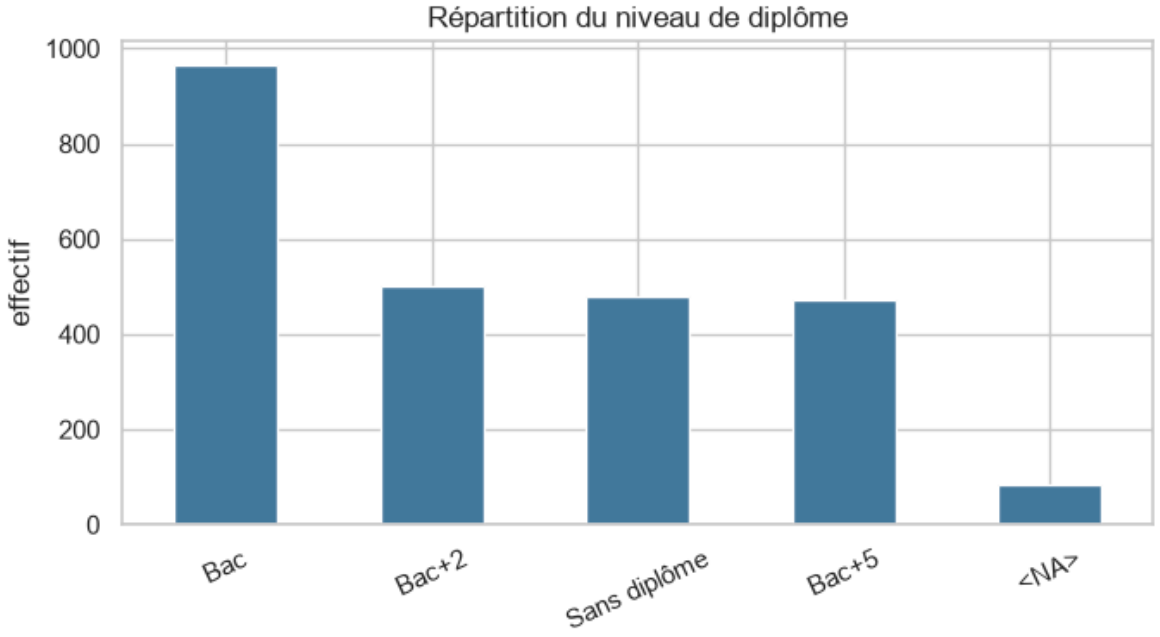
Variables numériques :

	count	mean	std	min	25%	50%	75%	max
age	2378.0	40.566442	13.225242	18.0	29.0	41.0	52.0	63.0
anciennete_poste_ans	2500.0	2.971600	2.979548	0.0	0.8	2.0	4.1	22.6
est_allocataire	2456.0	0.595684	0.490859	0.0	0.0	1.0	1.0	1.0
nationalite_hors_ue	2500.0	0.118000	0.322673	0.0	0.0	0.0	0.0	1.0
classe_retour_emploi	2500.0	0.806400	0.719671	0.0	0.0	1.0	1.0	2.0



Synthèse des variables catégorielles (cardinalité + manquants) :

	variable	nb_modalites	pct_manquants
0	usager_id	2500	0.00
3	code_insee_commune	2407	0.00
2	code_rome_vise	50	0.00
4	synthese_entretien	9	3.24
1	niveau_diplome	4	3.36



3.4 Variable cible : équilibre des classes

Mesure de la distribution de `classe_retour_emploi` pour évaluer un éventuel déséquilibre de classes.

In [7]:
```
import matplotlib.pyplot as plt

target_col = "classe_retour_emploi"
assert target_col in df.columns, f"Colonne cible absente : {target_col}"

target_dist = (
    df[target_col]
    .value_counts(dropna=False)
    .rename_axis(target_col)
```

```
    .to_frame("effectif")
    .sort_index()
)
target_dist["part_pct"] = (target_dist["effectif"] / target_dist["effectif"].sum

print("Distribution de la variable cible :")
display(target_dist)

fig, ax = plt.subplots(figsize=(6, 4))
target_dist["effectif"].plot(kind="bar", ax=ax, color=["#8ecae6", "#219ebc", "#0
ax.set_title("Distribution de la cible classe_retour_emploi")
ax.set_xlabel("classe")
ax.set_ylabel("effectif")
ax.tick_params(axis="x", rotation=0)
plt.tight_layout()
plt.show()

majority_pct = float(target_dist["part_pct"].max())
print(f"Classe majoritaire : {majority_pct:.2f}% du dataset")
```

```
Distribution de la variable cible :
```

classe_retour_emploi	effectif	part_pct
0	936	37.44
1	1112	44.48
2	452	18.08




```
Classe majoritaire : 44.48% du dataset
```

3.5 Relations entre variables et cible

Lecture des corrélations numériques et de croisements simples entre variables clés et cible.

In [8]:
```
import matplotlib.pyplot as plt

target_col = "classe_retour_emploi"
num_cols = df.select_dtypes(include=["number"]).columns.tolist()

# Corrélations numériques (hors auto-corrélation cible)
if target_col in num_cols:
    corr_target = (
        df[num_cols]
        .corr(numeric_only=True)[target_col]
        .drop(labels=[target_col])
        .sort_values(key=lambda s: s.abs(), ascending=False)
        .to_frame("corr_avec_cible")
    )
    print("Corrélations avec la cible (ordre décroissant en valeur absolue) :")
    display(corr_target)

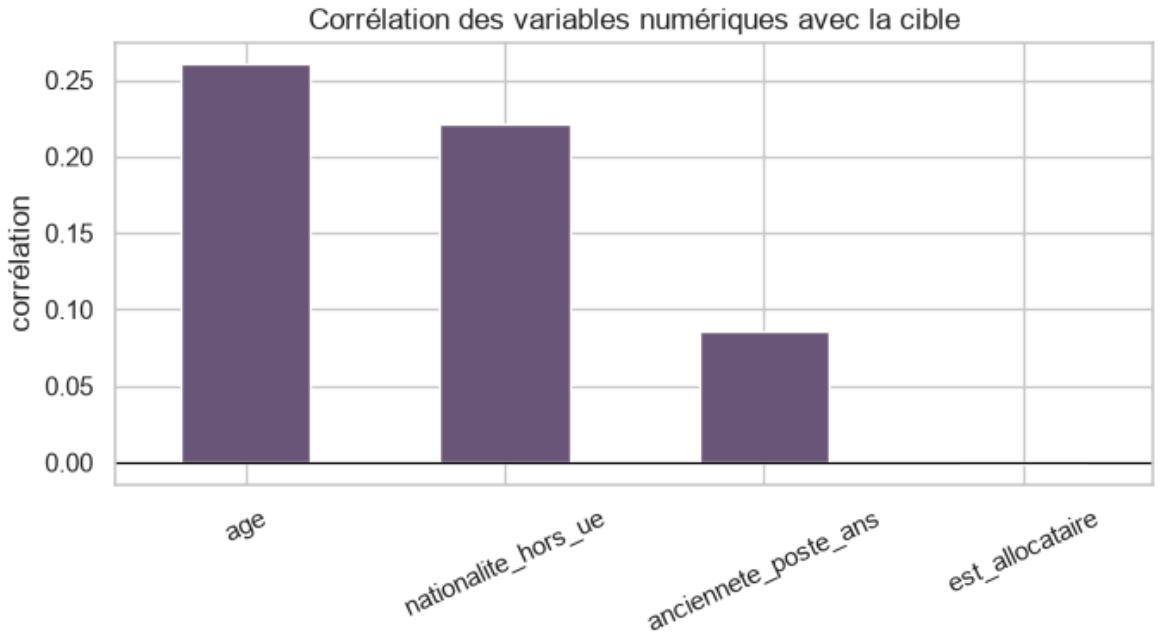
    fig, ax = plt.subplots(figsize=(7, 4))
    corr_target["corr_avec_cible"].plot(kind="bar", ax=ax, color="#6d597a")
    ax.set_title("Corrélation des variables numériques avec la cible")
    ax.set_ylabel("corrélation")
    ax.axhline(0, color="black", linewidth=0.8)
    ax.tick_params(axis="x", rotation=25)
    plt.tight_layout()
    plt.show()

# Croisements simples avec variables clés
for col in ["est_allocataire", "nationalite_hors_ue", "niveau_diplome"]:
    if col in df.columns:
        ct = pd.crosstab(df[col], df[target_col], normalize="index").round(3)
        print(f"\nRépartition de la cible conditionnelle à {col} :")
        display(ct)

        fig, ax = plt.subplots(figsize=(8, 4))
        ct.plot(kind="bar", stacked=True, ax=ax, colormap="viridis")
        ax.set_title(f"Répartition conditionnelle de la cible selon {col}")
        ax.set_xlabel(col)
        ax.set_ylabel("proportion")
        ax.legend(title="classe", bbox_to_anchor=(1.02, 1), loc="upper left")
        ax.tick_params(axis="x", rotation=25)
        plt.tight_layout()
        plt.show()
```

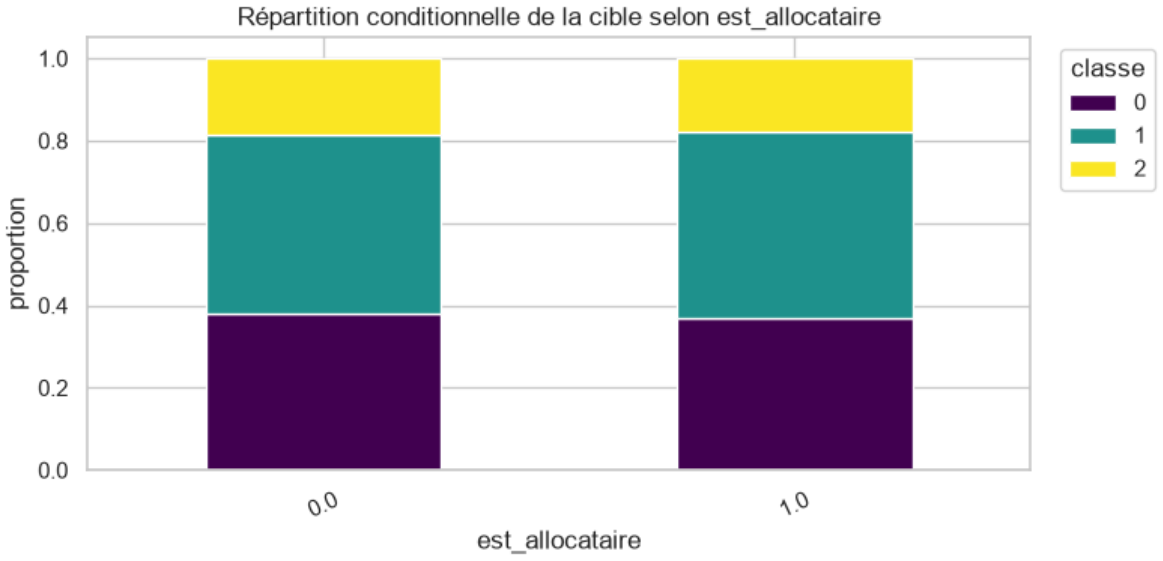
```
Corrélations avec la cible (ordre décroissant en valeur absolue) :
```

	corr_avec_cible
age	0.260681
nationalite_hors_ue	0.220764
anciennete_poste_ans	0.085816
est_allocataire	-0.001211



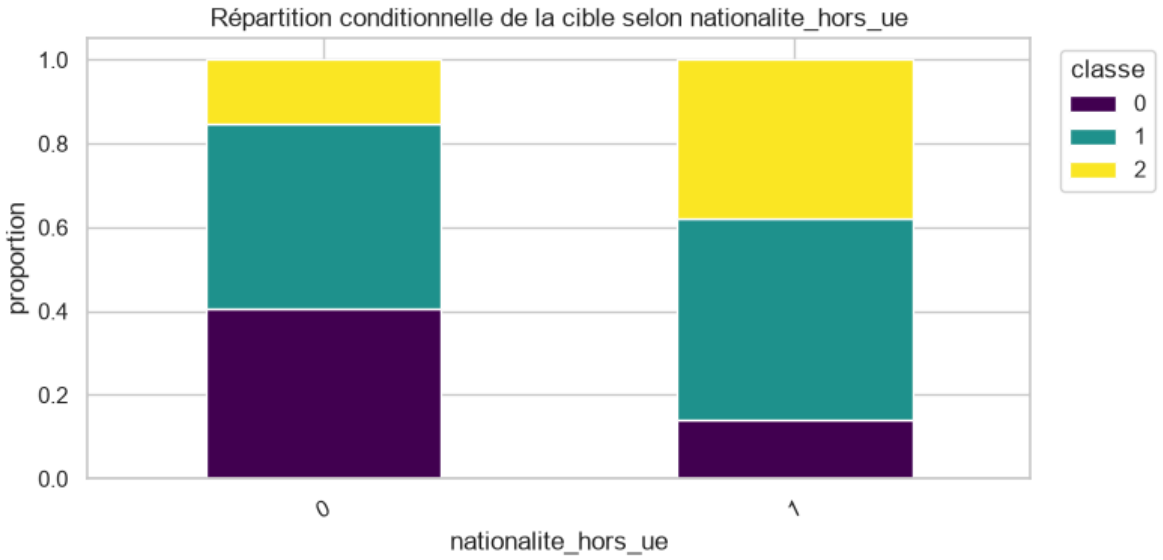
Répartition de la cible conditionnelle à est_allocataire :

classe_retour_emploi	0	1	2
est_allocataire			
0.0	0.379	0.435	0.186
1.0	0.371	0.452	0.177



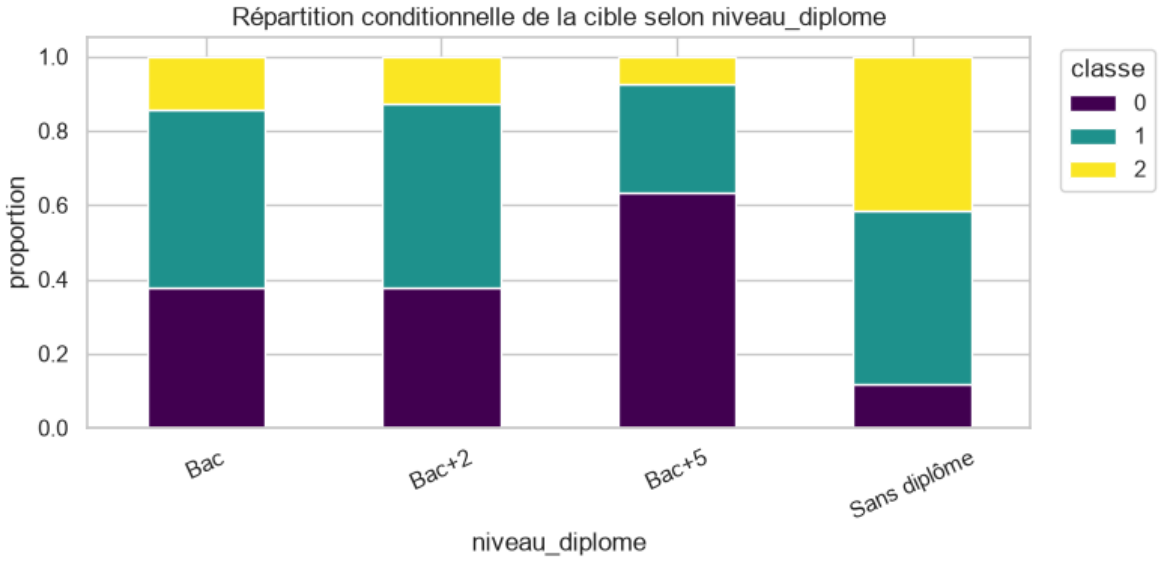
Répartition de la cible conditionnelle à nationalite_hors_ue :

classe_retour_emploi	0	1	2
nationalite_hors_ue			
0	0.406	0.440	0.154
1	0.139	0.481	0.380



Répartition de la cible conditionnelle à niveau_diplome :

classe_retour_emploi	0	1	2
niveau_diplome			
Bac	0.377	0.481	0.142
Bac+2	0.377	0.495	0.128
Bac+5	0.635	0.293	0.072
Sans diplôme	0.119	0.467	0.414



3.6 Analyse des biais et variables sensibles

Analyse des variables du dataset sous l'angle de la gouvernance et de la non-discrimination :

- `nationalite_hors_ue` **(variable sensible directe)** : présente un écart marqué d'orientation (`38,0 %` en classe 2 pour les ressortissants hors UE contre `15,4 %` pour les ressortissants UE). Pour garantir l'équité de traitement et respecter le cadre juridique du service public, cette variable est exclue du modèle de production

(scénario S2). Elle est conservée uniquement dans le scénario S1 à titre de comparaison expérimentale.

- `age` **(variable socio-professionnelle à surveiller)** : l'âge est directement pertinent pour calibrer l'accompagnement renforcé. Il est maintenu comme variable numérique dans le modèle, mais soumis à un suivi d'équité par tranche d'âge (ex. seniors, jeunes) afin d'éviter tout biais indirect d'éviction.
- `est_allocataire` et `departement_insee` **(proxys socio-territoriaux)** : présentent des écarts plus modérés (`18,6 %` vs `17,7 %` de classe 2 pour le statut allocataire). Le regroupement par département atténue la granularité trop fine de la commune.
- `synthese_entretien` **(texte libre)** : constitue un vecteur potentiel de biais de rédaction ou de stéréotypes exprimés par le conseiller, nécessitant une vigilance lors de l'interprétation des importances de mots.

Conclusion : cet état des lieux justifie le choix de conception de S2 (exclusion de la nationalité) et l'instauration d'un suivi régulier des performances par tranche d'âge et catégorie de diplôme.

3.7 Synthèse de l'analyse exploratoire (EDA)

Principaux enseignements :

1. **Volume et complétude** : 2 500 dossiers exploitables. Les valeurs manquantes sont modérées (`age` 4,88 %, `niveau_diplome` 3,36 %, `synthese_entretien` 3,24 %, `est_allocataire` 1,76 %) et gérées par imputation.
2. **Distribution de la cible** : déséquilibre modéré (`classe 1` = 44,5 %, `classe 0` = 37,4 %, `classe 2` = 18,1 %). Le F1 macro et le rappel de la classe 2 seront nos métriques phares.
3. **Signaux de discrimination et proxys** : écart significatif sur `nationalite_hors_ue` , justifiant son retrait du modèle de production. L'âge et le diplôme apportent une forte valeur explicative et sont conservés sous surveillance.
4. **Qualité du canal texte** : le champ `synthese_entretien` comporte seulement 9 formulations distinctes récurrentes. Ce manque de diversité limite l'apport de la vectorisation NLP et accroît le risque de sur-apprentissage sur des verbatims stéréotypés.

Implications pour le pipeline (sections 4 à 8) :

- Imputation médiane/mode robuste au sein des pipelines par type de variable.
- Encodage ordinal explicite pour le diplôme et one-hot pour les catégories nominales.
- Définition du scénario S2 (sans nationalité) comme candidat unique pour la mise en production.
- Mise en place d'une politique d'abstention (seuil de confiance + relecture humaine) pour sécuriser les cas ambigus et les erreurs critiques `2 -> 0` .

---

4. Préparation des données

Cette section construit un pipeline de préprocessing reproductible, compatible avec les quatre scénarios de comparaison. Le protocole suit une règle stricte anti-fuite : séparation train/test avant tout ajustement de transformation.

4.1 Train/Test Split (stratifié)

Objectif : figer un découpage d'évaluation propre, sans fuite de données, et préparer les listes de variables pour le pipeline de préprocessing.

In [9]:
```
from sklearn.model_selection import train_test_split

target_col = "classe_retour_emploi"
text_feature = "synthese_entretien"
id_cols = ["usager_id"]

assert target_col in df.columns, f"Cible absente: {target_col}"

# Séparation X/y
X = df.drop(columns=[target_col]).copy()
y = df[target_col].copy()

# Extraction du département depuis le code INSEE commune
if "code_insee_commune" in X.columns:
    insee_clean = (
        X["code_insee_commune"]
        .astype("string")
        .fillna("")
        .str.strip()
        .str.upper()
        .str.replace(r"[^0-9A-Z]", "", regex=True)
        .str.zfill(5)
    )
    X["departement_insee"] = insee_clean.str[:2].replace({"": pd.NA})

# On exclut les identifiants techniques et la commune brute du modèle
excluded_from_model = [c for c in id_cols + ["code_insee_commune"] if c in X.col
X_model = X.drop(columns=excluded_from_model, errors="ignore")

# Listes explicites: le diplôme est ordonné, les autres catégories sont nominale
numeric_candidates = ["age", "anciennete_poste_ans"]
ordinal_candidates = ["niveau_diplome"]
categorical_candidates = [
    "code_rome_vise",
    "est_allocataire",
    "nationalite_hors_ue",
    "departement_insee",
]

num_features = [c for c in numeric_candidates if c in X_model.columns]
ordinal_features = [c for c in ordinal_candidates if c in X_model.columns]
cat_features = [c for c in categorical_candidates if c in X_model.columns]

unknown_features = sorted(
    set(X_model.columns)
```

```
    - set(num_features)
    - set(ordinal_features)
    - set(cat_features)
    - {text_feature}
)

assert text_feature in X_model.columns, f"Feature texte absente: {text_feature}"
assert "departement_insee" in X_model.columns, "Feature engineering INSEE absent
assert "code_insee_commune" not in X_model.columns, "La commune brute doit être
assert "niveau_diplome" not in cat_features, "niveau_diplome doit être traité en
if "est_allocataire" in X_model.columns:
    assert "est_allocataire" in cat_features, "est_allocataire doit être traité
assert not unknown_features, f"Features non mappées: {unknown_features}"
assert len(num_features) + len(ordinal_features) + len(cat_features) + 1 == X_mo

X_train, X_test, y_train, y_test = train_test_split(
    X_model,
    y,
    test_size=0.2,
    random_state=RANDOM_STATE,
    stratify=y,
    shuffle=True,
)

print(f"Taille totale   : {len(df)}")
print(f"Train/Test      : {X_train.shape} / {X_test.shape}")
print(f"Num features    : {len(num_features)} -> {num_features}")
print(f"Ord features    : {len(ordinal_features)} -> {ordinal_features}")
print(f"Cat features    : {len(cat_features)} -> {cat_features}")
print(f"Text feature    : {text_feature}")
print(f"Colonnes exclues: {excluded_from_model}")

print("\nFeature engineering INSEE:")
print("- nouvelle feature: departement_insee (2 premiers caractères du code INSE
if "departement_insee" in X_model.columns:
    print("- top 5 départements:")
    display(X_model["departement_insee"].value_counts(dropna=False).head(5).to_f

print("\nDistribution cible (global / train / test) :")
dist_global = y.value_counts(normalize=True).sort_index().rename("global")
dist_train = y_train.value_counts(normalize=True).sort_index().rename("train")
dist_test = y_test.value_counts(normalize=True).sort_index().rename("test")
display(pd.concat([dist_global, dist_train, dist_test], axis=1).round(4))

assert (len(X_train) + len(X_test)) == len(X_model), "Split incomplet"
assert set(y_train.unique()) == set(y_test.unique()) == set(y.unique()), "Perte
```

```
Taille totale   : 2500
Train/Test      : (2000, 8) / (500, 8)
Num features    : 2 -> ['age', 'anciennete_poste_ans']
Ord features    : 1 -> ['niveau_diplome']
Cat features    : 4 -> ['code_rome_vise', 'est_allocataire', 'nationalite_hors_u
e', 'departement_insee']
Text feature    : synthese_entretien
Colonnes exclues: ['usager_id', 'code_insee_commune']

Feature engineering INSEE:
- nouvelle feature: departement_insee (2 premiers caractères du code INSEE)
- top 5 départements:
```

	effectif
departement_insee	
21	67
62	62
57	60
02	59
27	52

```
Distribution cible (global / train / test) :
```

	global	train	test
classe_retour_emploi			
0	0.3744	0.3745	0.374
1	0.4448	0.4445	0.446
2	0.1808	0.1810	0.180

4.2 Pipeline de préprocessing (numérique, ordinal, catégoriel, texte)

Objectif : définir un préprocessing unique, reproductible et sans fuite, qui sera branché ensuite aux modèles en section 5.

`niveau_diplome` est traité séparément avec un encodage ordinal explicite: `Sans diplôme < Bac < Bac+2 < Bac+5`. Cet ordre reprend une hiérarchie métier déclarée; il ne signifie pas que le diplôme est une cause du retour à l'emploi.

Les variables sans ordre naturel (`code_rome_vise` , `est_allocataire` , `nationalite_hors_ue` , `departement_insee`) restent en one-hot encoding dans les scénarios de comparaison. Pour la production, S2 exclut `nationalite_hors_ue` .

La colonne `code_insee_commune` est conservée dans les données sources pour la traçabilité, mais explicitement exclue des features de modélisation.

In [10]:
```
from sklearn.compose import ColumnTransformer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import FunctionTransformer, OneHotEncoder, OrdinalEnc

# Pipeline numérique : imputation robuste + mise à l'échelle
numeric_pipeline = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ]
)

# Pipeline ordinal : ordre métier explicite pour le niveau de diplôme.
```

```python
ordinal_pipeline = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="most_frequent")),
        (
            "encoder",
            OrdinalEncoder(
                categories=[["Sans diplôme", "Bac", "Bac+2", "Bac+5"]],
                handle_unknown="use_encoded_value",
                unknown_value=-1,
            ),
        ),
    ]
)

# Pipeline catégoriel nominal : imputation + one-hot encoding.
categorical_pipeline = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("encoder", OneHotEncoder(handle_unknown="ignore")),
    ]
)

# Pipeline texte : sécuriser les NaN puis vectorisation TF-IDF
text_pipeline = Pipeline(
    steps=[
        (
            "to_text",
            FunctionTransformer(
                lambda s: s.squeeze().fillna(""),
                validate=False,
            ),
        ),
        ("tfidf", TfidfVectorizer(max_features=500, ngram_range=(1, 2))),
    ]
)

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_pipeline, num_features),
        ("ord", ordinal_pipeline, ordinal_features),
        ("cat", categorical_pipeline, cat_features),
        ("txt", text_pipeline, [text_feature]),
    ],
    remainder="drop",
)

# Fit uniquement sur train, puis transformation train/test
X_train_prepared = preprocessor.fit_transform(X_train)
X_test_prepared = preprocessor.transform(X_test)

print("Préprocessing prêt.")
print(f"Train transformé : {X_train_prepared.shape}")
print(f"Test transformé  : {X_test_prepared.shape}")
print(f"Type matrice train : {type(X_train_prepared).__name__}")
```

```
Préprocessing prêt.
Train transformé : (2000, 289)
Test transformé  : (500, 289)
Type matrice train : csr_matrix
```

4.3 Scénarios de jeux de données

Le projet compare **4 scénarios** avec un protocole constant pour évaluer performance et robustesse éthique.

Scénario	Variables conservées	Justification
S1 - Multimodal complet	Numériques + catégorielles + texte (dont `nationalite_hors_ue`)	Baseline de performance maximale attendue
S2 - Multimodal sans variable sensible	S1 sans `nationalite_hors_ue`	Mesurer l'impact de la variable sensible sur performance/biais
S3 - Texte seul	`synthese_entretien` uniquement	Évaluer la valeur prédictive intrinsèque du verbatim conseiller
S4 - Tabulaire seul	Numériques + catégorielles (sans texte)	Comparer la contribution du texte au gain de performance

4.4 Synthèse préparation

- Le split stratifié est figé en amont (section 4.1) pour garantir des comparaisons équitables entre scénarios.
- Un pipeline de préprocessing unifié (section 4.2) est réutilisé avec adaptation des colonnes selon le scénario.
- La comparaison des 4 scénarios sera réalisée en section 5 puis arbitrée en section 6 sur un compromis performance / risque de biais / robustesse.

In [11]:
```
# 4.3bis - Préparation opérationnelle des 4 scénarios

from copy import deepcopy


def make_preprocessor(num_feats, ord_feats, cat_feats, use_text=True):
    transformers = [
        ("num", deepcopy(numeric_pipeline), num_feats),
        ("ord", deepcopy(ordinal_pipeline), ord_feats),
        ("cat", deepcopy(categorical_pipeline), cat_feats),
    ]
    if use_text:
        transformers.append(("txt", deepcopy(text_pipeline), [text_feature]))

    return ColumnTransformer(transformers=transformers, remainder="drop")


scenarios = {
    "S1_multimodal_complet": {
        "drop_cols": [],
        "use_text": True,
        "description": "Toutes les variables + texte (inclut variable sensible)"
    },
    "S2_sans_variable_sensible": {
        "drop_cols": ["nationalite_hors_ue"],
        "use_text": True,
```

```
        "description": "Multimodal sans nationalite_hors_ue",
    },
    "S3_texte_seul": {
        "drop_cols": [c for c in X_train.columns if c != text_feature],
        "use_text": True,
        "description": "Uniquement le verbatim synthese_entretien",
    },
    "S4_tabulaire_seul": {
        "drop_cols": [text_feature],
        "use_text": False,
        "description": "Variables tabulaires uniquement (sans texte)",
    },
}

scenario_shapes = []
for name, cfg in scenarios.items():
    Xtr = X_train.drop(columns=cfg["drop_cols"], errors="ignore")
    Xte = X_test.drop(columns=cfg["drop_cols"], errors="ignore")

    num_feats = [c for c in num_features if c in Xtr.columns]
    ord_feats = [c for c in ordinal_features if c in Xtr.columns]
    cat_feats = [c for c in cat_features if c in Xtr.columns]

    allowed_extra = {text_feature} if cfg["use_text"] else set()
    unknown_scenario_features = sorted(
        set(Xtr.columns)
        - set(num_feats)
        - set(ord_feats)
        - set(cat_feats)
        - allowed_extra
    )
    assert not unknown_scenario_features, f"Features non mappées pour {name}: {u

    pp = make_preprocessor(num_feats, ord_feats, cat_feats, use_text=cfg["use_te
    Xtr_t = pp.fit_transform(Xtr)
    Xte_t = pp.transform(Xte)

    scenario_shapes.append({
        "scenario": name,
        "description": cfg["description"],
        "n_features_output": Xtr_t.shape[1],
        "train_shape": Xtr_t.shape,
        "test_shape": Xte_t.shape,
    })

scenario_summary = pd.DataFrame(scenario_shapes)
display(scenario_summary)
```

	scenario	description	n_features_output	train_shape	test_shape
0	S1_multimodal_complet	Toutes les variables + texte (inclut variable ...	289	(2000, 289)	(500, 289)
1	S2_sans_variable_sensible	Multimodal sans nationalite_hors_ue	287	(2000, 287)	(500, 287)
2	S3_texte_seul	Uniquement le verbatim synthese_entretien	136	(2000, 136)	(500, 136)
3	S4_tabulaire_seul	Variables tabulaires uniquement (sans texte)	153	(2000, 153)	(500, 153)

4.5 Assertions de qualité

Cette étape valide, via des `assert`, que les données préparées sont cohérentes avant modélisation. Une assertion en échec signale un point à corriger avant de poursuivre.

Ces contrôles servent de base à une future suite de tests automatisés.

In [12]:
```
# Pré-requis minimaux
required_objects = [
    "df", "X_train", "X_test", "y_train", "y_test",
    "preprocessor", "X_train_prepared", "X_test_prepared", "scenario_summary",
    "target_col", "text_feature", "num_features", "ordinal_features", "cat_featu
    "scenarios", "excluded_from_model", "X_model",
]
for obj in required_objects:
    assert obj in globals(), f"Objet manquant pour la section 4: {obj}"

# 1) Cohérence cible / dataframe
assert target_col in df.columns, f"Cible absente du dataset: {target_col}"
assert df[target_col].isna().sum() == 0, "Manquants détectés dans la cible"
assert set(df[target_col].dropna().unique()).issubset({0, 1, 2}), "Classes inatt

# 2) Cohérence split
assert len(X_train) > 0 and len(X_test) > 0, "Split vide"
assert (len(X_train) + len(X_test)) == len(X_model), "Split train/test incomplet
assert set(y_train.unique()) == set(y_test.unique()) == set(df[target_col].uniqu

# 3) Cohérence features
assert text_feature in X_train.columns, f"Feature texte absente: {text_feature}"
assert text_feature not in num_features, "La feature texte ne doit pas être dans
assert text_feature not in ordinal_features, "La feature texte ne doit pas être
assert text_feature not in cat_features, "La feature texte est gérée séparément,
assert "niveau_diplome" in ordinal_features, "niveau_diplome doit être traité en
assert "niveau_diplome" not in cat_features, "niveau_diplome ne doit pas être en
for col in excluded_from_model:
    assert col not in X_train.columns, f"Colonne technique non exclue: {col}"
assert set(num_features).issubset(set(X_train.columns)), "num_features contient
assert set(ordinal_features).issubset(set(X_train.columns)), "ordinal_features c
assert set(cat_features).issubset(set(X_train.columns)), "cat_features contient

# 4) Cohérence préprocessing global
```

```python
assert X_train_prepared.shape[0] == X_train.shape[0], "Nombre de lignes train mo
assert X_test_prepared.shape[0] == X_test.shape[0], "Nombre de lignes test modif
assert X_train_prepared.shape[1] == X_test_prepared.shape[1], "Dimension feature
assert X_train_prepared.shape[1] > 0, "Aucune feature après transformation"

# 5) Cohérence scénarios
expected_scenarios = {
    "S1_multimodal_complet",
    "S2_sans_variable_sensible",
    "S3_texte_seul",
    "S4_tabulaire_seul",
}
assert set(scenarios.keys()) == expected_scenarios, "Scénarios manquants ou inat
assert scenario_summary.shape[0] == 4, "Le tableau scénario doit contenir 4 lign
assert set(scenario_summary["scenario"]) == expected_scenarios, "Noms de scénari
assert (scenario_summary["n_features_output"] > 0).all(), "Un scénario produit 0

nfeat = scenario_summary.set_index("scenario")["n_features_output"].to_dict()
assert nfeat["S2_sans_variable_sensible"] <= nfeat["S1_multimodal_complet"], "S2
assert nfeat["S3_texte_seul"] < nfeat["S1_multimodal_complet"], "S3 (texte seul)
assert nfeat["S4_tabulaire_seul"] < nfeat["S1_multimodal_complet"], "S4 (sans te

print("Toutes les assertions section 4 sont validées.")
print("Section 4 clôturée: données préparées, scénarios traçables, protocole ant
```

```
Toutes les assertions section 4 sont validées.
Section 4 clôturée: données préparées, scénarios traçables, protocole anti-fuite
confirmé.
```

---

5. Modélisation et entraînement

Cette section compare plusieurs modèles avec validation croisée, puis sélectionne un modèle final sur des critères explicites : performance, robustesse et lisibilité métier. Le jeu de test reste réservé à l'évaluation finale.

5.0 Familles de modèles candidates

Avant de sélectionner des modèles précis, cette section compare les grandes familles de solutions pour expliciter le périmètre technique retenu.

Famille	Adaptée au problème ?	Coût total (entraînement + inférence)	Explicabilité	Dépendance fournisseur	Décision (retenue / écartée + motif)
ML classique (scikit-learn, XGBoost)	Oui	Faible à moyen	Bonne à moyenne	Faible	**Retenue**. Bon compromis pour un dataset de taille modérée (2 500 lignes), un besoin de classification supervisée multiclasse et une mise en

Famille	Adaptée au problème ?	Coût total (entraînement + inférence)	Explicabilité	Dépendance fournisseur	Décision (retenue / écartée + motif)
					production sobre. Permet des temps d'entraînement raisonnables, une inférence rapide et une explicabilité suffisante pour un contexte service public.
Deep Learning (Keras / PyTorch)	Peu adaptée	Élevé	Faible à moyenne	Faible	**Écartée**. Le volume de données est trop limité pour justifier un réseau profond robuste. Le surcoût d'entraînement, la complexité de réglage et le déficit d'explicabilité ne sont pas proportionnés au gain attendu.
SLM local (Ollama, modèles <= 7B)	Peu adaptée	Élevé	Faible	Faible à moyenne	**Écartée**. Ces modèles peuvent aider sur des tâches de génération ou de résumé, mais ils ne constituent pas ici la solution la plus simple ni la plus fiable pour une classification supervisée tabulaire + texte avec exigence de traçabilité.
LLM API + RAG	Non adaptée	Élevé et récurrent	Faible	Élevée	**Écartée**. Le cas d'usage n'est pas un problème de recherche documentaire ou de génération assistée. Le recours à une API externe ajoute des coûts, des enjeux de confidentialité et une dépendance

Famille	Adaptée au problème ?	Coût total (entraînement + inférence)	Explicabilité	Dépendance fournisseur	Décision (retenue / écartée + motif)
					fournisseur difficiles à justifier.
Architecture agentique (multi-LLM, tools)	Non adaptée	Très élevé	Faible	Élevée	**Écartée**. Architecture disproportionnée pour un pipeline de prédiction structuré. Elle complexifie fortement l'industrialisation, la gouvernance et l'auditabilité sans répondre au besoin principal de classification reproductible.

5.0.1 Synthèse familles

La famille **ML classique** est la plus adaptée au cadre de ce projet : peu de données, objectif de classification bien défini, besoin de rapidité d'inférence et exigence minimale d'explicabilité.

Les autres familles ont été écartées non parce qu'elles sont inutiles en général, mais parce qu'elles augmentent ici la complexité, le coût et le risque de gouvernance sans avantage clair sur la performance attendue.

La contrainte la plus déterminante est donc le **bon ratio entre performance, lisibilité métier et sobriété d'industrialisation**.

5.1 Modèles candidats

Les modèles retenus pour la comparaison couvrent plusieurs compromis performance/coût/interprétabilité :

Modèle	Famille	Rôle dans l'analyse	Coût entraînement / inférence	Explicabilité
LogisticRegression	linéaire	baseline rapide et stable	faible / faible	élevée
RandomForestClassifier	ensemble bagging	baseline robuste sur tabulaire	moyen / faible	moyenne
XGBoost	gradient boosting	candidat performant multimodal	moyen-élevé / faible	moyenne

Modèle	Famille	Rôle dans l'analyse	Coût entraînement / inférence	Explicabilité
LightGBM	gradient boosting	alternative performante et rapide	moyen / faible	moyenne

Ces quatre modèles sont évalués avec le même protocole de validation croisée stratifiée.

5.2 Entraînement & validation croisée stratifiée

In [13]:
```
from time import perf_counter

import numpy as np
import pandas as pd
from sklearn.base import clone
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold, cross_validate
from sklearn.pipeline import Pipeline
from sklearn.metrics import make_scorer, recall_score
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier

# Scorer métier : recall de la classe critique (2)
def recall_class_2(y_true, y_pred):
    return recall_score(y_true, y_pred, labels=[2], average="macro", zero_divisi


scoring = {
    "accuracy": "accuracy",
    "f1_macro": "f1_macro",
    "recall_class_2": make_scorer(recall_class_2),
    "roc_auc_ovr_macro": "roc_auc_ovr",
}

candidate_models = {
    "logreg": LogisticRegression(max_iter=2000, class_weight="balanced", random_
    "random_forest": RandomForestClassifier(
        n_estimators=300, max_depth=None, min_samples_leaf=2,
        class_weight="balanced", n_jobs=-1, random_state=RANDOM_STATE,
    ),
    "xgboost": XGBClassifier(
        n_estimators=350, max_depth=6, learning_rate=0.05, subsample=0.9,
        colsample_bytree=0.9, objective="multi:softprob", num_class=3,
        eval_metric="mlogloss", random_state=RANDOM_STATE, n_jobs=-1,
    ),
    "lightgbm": LGBMClassifier(
        n_estimators=350, learning_rate=0.05, num_leaves=31, subsample=0.9,
        colsample_bytree=0.9, objective="multiclass", class_weight="balanced",
        random_state=RANDOM_STATE, n_jobs=-1, verbosity=-1,
    ),
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

# Préparation des jeux par scénario avec les familles numérique, ordinale et nom
```

```
scenario_data = {}
for scen_name, cfg in scenarios.items():
    Xtr = X_train.drop(columns=cfg["drop_cols"], errors="ignore")
    Xte = X_test.drop(columns=cfg["drop_cols"], errors="ignore")

    num_feats = [c for c in num_features if c in Xtr.columns]
    ord_feats = [c for c in ordinal_features if c in Xtr.columns]
    cat_feats = [c for c in cat_features if c in Xtr.columns]

    allowed_extra = {text_feature} if cfg["use_text"] else set()
    unknown_scenario_features = sorted(
        set(Xtr.columns) - set(num_feats) - set(ord_feats) - set(cat_feats) - al
    )
    assert not unknown_scenario_features, f"Features non mappées pour {scen_name

    pp = make_preprocessor(num_feats, ord_feats, cat_feats, use_text=cfg["use_te
    scenario_data[scen_name] = {
        "X_train": Xtr,
        "X_test": Xte,
        "preprocessor": pp,
    }

cv_rows = []
for scen_name, scen_obj in scenario_data.items():
    for model_name, model in candidate_models.items():
        pipe = Pipeline([
            ("prep", clone(scen_obj["preprocessor"])),
            ("clf", clone(model)),
        ])

        started_at = perf_counter()
        scores = cross_validate(
            pipe,
            scen_obj["X_train"],
            y_train,
            cv=cv,
            scoring=scoring,
            n_jobs=1,
            return_train_score=False,
            error_score="raise",
        )
        duration = perf_counter() - started_at

        cv_rows.append({
            "scenario": scen_name,
            "model": model_name,
            "cv_accuracy_mean": float(np.mean(scores["test_accuracy"])),
            "cv_accuracy_std": float(np.std(scores["test_accuracy"])),
            "cv_f1_macro_mean": float(np.mean(scores["test_f1_macro"])),
            "cv_f1_macro_std": float(np.std(scores["test_f1_macro"])),
            "cv_recall_class2_mean": float(np.mean(scores["test_recall_class_2"]
            "cv_recall_class2_std": float(np.std(scores["test_recall_class_2"]))
            "cv_roc_auc_ovr_macro_mean": float(np.mean(scores["test_roc_auc_ovr_
            "cv_roc_auc_ovr_macro_std": float(np.std(scores["test_roc_auc_ovr_ma
            "cv_runtime_s": round(duration, 2),
        })

cv_results = pd.DataFrame(cv_rows).sort_values(
    by=["cv_f1_macro_mean", "cv_recall_class2_mean"],
    ascending=False,
```

```
).reset_index(drop=True)

display(cv_results)
```

	scenario	model	cv_accuracy_mean	cv_accuracy_std	cv_f1_macr
0	S1_multimodal_complet	xgboost	0.7295	0.018868	0
1	S1_multimodal_complet	lightgbm	0.7015	0.020408	0
2	S2_sans_variable_sensible	xgboost	0.7020	0.020211	0
3	S4_tabulaire_seul	random_forest	0.6900	0.021213	0
4	S1_multimodal_complet	logreg	0.6805	0.016233	0
5	S2_sans_variable_sensible	lightgbm	0.6810	0.015859	0
6	S4_tabulaire_seul	xgboost	0.6915	0.016553	0
7	S2_sans_variable_sensible	logreg	0.6760	0.013565	0
8	S1_multimodal_complet	random_forest	0.6680	0.022771	0
9	S2_sans_variable_sensible	random_forest	0.6605	0.020821	0
10	S4_tabulaire_seul	lightgbm	0.6595	0.029043	0
11	S3_texte_seul	logreg	0.6515	0.019013	0
12	S3_texte_seul	random_forest	0.6515	0.019013	0
13	S3_texte_seul	xgboost	0.6515	0.019013	0
14	S3_texte_seul	lightgbm	0.6515	0.019013	0
15	S4_tabulaire_seul	logreg	0.5395	0.017421	0

5.3 Métriques retenues et justification

Métriques suivies pour la classification multiclasse :

- F1 macro (critère principal de classement, équilibré entre les trois classes) ;
- recall de la classe critique ;
- ROC-AUC macro One-vs-Rest (qualité du classement probabiliste) ;
- matrice de confusion ;
- accuracy en indicateur complémentaire.

Le ROC-AUC est utile pour vérifier la qualité des probabilités produites, mais il ne remplace pas le F1 macro ni le recall de la classe 2. Un modèle peut bien classer ses probabilités tout en générant trop d'erreurs critiques après application de la règle de décision.

La sélection finale est donc justifiée au regard des enjeux métier, pas uniquement sur un score global.

5.4 Comparaison des modèles : tableau synthétique

In [14]:

```python
assert "cv_results" in globals(), "Exécuter la cellule 5.2 avant la comparaison"

comparison_cols = [
    "scenario",
    "model",
    "cv_f1_macro_mean",
    "cv_recall_class2_mean",
    "cv_roc_auc_ovr_macro_mean",
    "cv_accuracy_mean",
    "cv_runtime_s",
]

comparison_table = cv_results[comparison_cols].copy()
comparison_table["rank"] = comparison_table["cv_f1_macro_mean"].rank(
    ascending=False, method="dense"
).astype(int)

comparison_table = comparison_table.sort_values(["rank", "cv_recall_class2_mean"

display(comparison_table.style.format({
    "cv_f1_macro_mean": "{:.3f}",
    "cv_recall_class2_mean": "{:.3f}",
    "cv_roc_auc_ovr_macro_mean": "{:.3f}",
    "cv_accuracy_mean": "{:.3f}",
    "cv_runtime_s": "{:.2f}",
}))

best_row = comparison_table.iloc[0]
best_scenario = best_row["scenario"]
best_model_name = best_row["model"]
print(f"Meilleur couple CV selon F1 macro: {best_scenario} + {best_model_name}")
print(f"ROC-AUC OVR macro associé: {best_row['cv_roc_auc_ovr_macro_mean']:.3f}")

# Benchmark de livraison : métriques test, latence p95 et proxy de coût CPU.
# Hypothèse de coût explicitée dans la section 6.1 : 0,05 EUR / vCPU-h.
from time import perf_counter
from sklearn.metrics import f1_score, recall_score

benchmark_specs = {
    "S1_multimodal_complet": ("xgboost", {}),
    "S2_sans_variable_sensible": ("xgboost", {
        "n_estimators": 300,
        "max_depth": 5,
        "learning_rate": 0.03,
        "subsample": 0.9,
        "colsample_bytree": 0.9,
    }),
    "S3_texte_seul": ("logreg", {}),
    "S4_tabulaire_seul": ("random_forest", {}),
}
benchmark_rows = []
for scenario_name, (model_name, params) in benchmark_specs.items():
    scenario = scenario_data[scenario_name]
    model = clone(candidate_models[model_name]).set_params(**params)
    benchmark_pipeline = Pipeline([
        ("prep", clone(scenario["preprocessor"])),
        ("clf", model),
    ])
    benchmark_pipeline.fit(scenario["X_train"], y_train)
```

```
    benchmark_pred = benchmark_pipeline.predict(scenario["X_test"])

    batch_times_ms = []
    for _ in range(30):
        started_at = perf_counter()
        benchmark_pipeline.predict(scenario["X_test"])
        batch_times_ms.append((perf_counter() - started_at) / len(scenario["X_te
    p95_ms = float(np.percentile(batch_times_ms, 95))
    cost_per_1000 = (p95_ms * 1000 / 1000) / 1000 / 3600 * 0.05

    benchmark_rows.append({
        "scenario": scenario_name,
        "model": model_name,
        "test_f1_macro": float(f1_score(y_test, benchmark_pred, average="macro")
        "test_recall_class2": float(recall_score(y_test, benchmark_pred, labels=
        "test_critical_2_to_0": int(((y_test.to_numpy() == 2) & (benchmark_pred
        "latency_p95_ms": p95_ms,
        "proxy_cost_eur_per_1000": cost_per_1000,
    })

scenario_benchmark = pd.DataFrame(benchmark_rows)
display(scenario_benchmark.style.format({
    "test_f1_macro": "{:.3f}",
    "test_recall_class2": "{:.3f}",
    "latency_p95_ms": "{:.3f}",
    "proxy_cost_eur_per_1000": "{:.2e}",
}))
```

	scenario	model	cv_f1_macro_mean	cv_recall_class2_mean	cv_rc
0	S1_multimodal_complet	xgboost	0.708	0.572	
1	S1_multimodal_complet	lightgbm	0.686	0.608	
2	S2_sans_variable_sensible	xgboost	0.681	0.558	
3	S4_tabulaire_seul	random_forest	0.672	0.589	
4	S1_multimodal_complet	logreg	0.665	0.669	
5	S2_sans_variable_sensible	lightgbm	0.664	0.591	
6	S4_tabulaire_seul	xgboost	0.664	0.467	
7	S2_sans_variable_sensible	logreg	0.660	0.674	
8	S1_multimodal_complet	random_forest	0.653	0.710	
9	S2_sans_variable_sensible	random_forest	0.645	0.694	
10	S4_tabulaire_seul	lightgbm	0.643	0.555	
11	S3_texte_seul	logreg	0.635	0.661	
12	S3_texte_seul	random_forest	0.635	0.661	
13	S3_texte_seul	xgboost	0.635	0.661	
14	S3_texte_seul	lightgbm	0.635	0.661	
15	S4_tabulaire_seul	logreg	0.526	0.566	

```
Meilleur couple CV selon F1 macro: S1_multimodal_complet + xgboost
ROC-AUC OVR macro associé: 0.874
```

	scenario	model	test_f1_macro	test_recall_class2	test_critical_2_t
0	S1_multimodal_complet	xgboost	0.720	0.511	
1	S2_sans_variable_sensible	xgboost	0.704	0.533	
2	S3_texte_seul	logreg	0.637	0.644	
3	S4_tabulaire_seul	random_forest	0.688	0.544	

5.5 Optimisation d'hyperparamètres du scénario de production

S1 est conservé comme référence expérimentale pour mesurer le gain de performance lié à `nationalite_hors_ue`. Il ne peut toutefois pas être retenu en production : une décision automatisée ne doit pas exploiter directement cette variable sensible.

L'optimisation est donc réalisée sur le meilleur modèle du scénario **S2**, qui conserve l'approche multimodale tout en excluant `nationalite_hors_ue`. Cette décision privilégie la conformité et la réduction du risque de discrimination directe; les autres variables sont ensuite surveillées comme proxys potentiels.

In [15]:
```
from sklearn.model_selection import ParameterGrid, cross_validate
from time import perf_counter

assert "comparison_table" in globals(), "Exécuter la section 5.4 avant cette cel

# S1 sert de référence expérimentale. S2 est le seul scénario éligible à la
# production car il exclut la variable sensible nationalite_hors_ue.
production_scenario = "S2_sans_variable_sensible"
production_candidates = comparison_table.loc[
    comparison_table["scenario"] == production_scenario
].sort_values(["rank", "cv_f1_macro_mean"], ascending=[True, False])

assert not production_candidates.empty, "Le scénario S2 est absent du tableau de
best_production_candidate = production_candidates.iloc[0]
selected_scenario = production_scenario
selected_model_name = best_production_candidate["model"]

param_grids = {
    "xgboost": {
        "n_estimators": [300, 450],
        "max_depth": [5, 7],
        "learning_rate": [0.03, 0.05],
        "subsample": [0.9],
        "colsample_bytree": [0.9],
    },
    "lightgbm": {
        "n_estimators": [300, 450],
        "num_leaves": [31, 63],
        "learning_rate": [0.03, 0.05],
        "subsample": [0.9],
        "colsample_bytree": [0.9],
    },
    "random_forest": {
        "n_estimators": [300, 500],
        "max_depth": [None, 16],
```

```
        "min_samples_leaf": [1, 2],
    },
    "logreg": {
        "C": [0.5, 1.0, 2.0],
    },
}

if selected_model_name not in param_grids:
    raise ValueError(f"Aucune grille définie pour {selected_model_name}")

X_tune = scenario_data[selected_scenario]["X_train"]
prep_tune = scenario_data[selected_scenario]["preprocessor"]
base_model = candidate_models[selected_model_name]
grid = list(ParameterGrid(param_grids[selected_model_name]))

tuning_rows = []
for candidate_index, params in enumerate(grid, start=1):
    model = clone(base_model).set_params(**params)
    pipe = Pipeline([
        ("prep", clone(prep_tune)),
        ("clf", model),
    ])

    started_at = perf_counter()
    scores = cross_validate(
        pipe,
        X_tune,
        y_train,
        cv=cv,
        scoring=scoring,
        n_jobs=1,
        return_train_score=False,
        error_score="raise",
    )
    elapsed = perf_counter() - started_at

    tuning_rows.append({
        "candidate_idx": candidate_index,
        "scenario": selected_scenario,
        "model": selected_model_name,
        "params": params,
        "cv_f1_macro_mean": float(np.mean(scores["test_f1_macro"])),
        "cv_recall_class2_mean": float(np.mean(scores["test_recall_class_2"])),
        "cv_roc_auc_ovr_macro_mean": float(np.mean(scores["test_roc_auc_ovr_macr
        "cv_accuracy_mean": float(np.mean(scores["test_accuracy"])),
        "cv_runtime_s": round(elapsed, 2),
    })

tuning_results = pd.DataFrame(tuning_rows).sort_values(
    by=["cv_recall_class2_mean", "cv_f1_macro_mean"],
    ascending=False,
).reset_index(drop=True)

display(tuning_results)

best_tuned_row = tuning_results.iloc[0]
selected_params = best_tuned_row["params"]

print("Sélection finale section 5.5 (S2 production + tuning):")
print(f"- scenario: {selected_scenario}")
```

```
print(f"- model: {selected_model_name}")
print(f"- params: {selected_params}")
print(f"- cv_recall_classe_2: {best_tuned_row['cv_recall_class2_mean']:.3f}")
print(f"- cv_f1_macro: {best_tuned_row['cv_f1_macro_mean']:.3f}")
print(f"- cv_roc_auc_ovr_macro: {best_tuned_row['cv_roc_auc_ovr_macro_mean']:.3f
```

	candidate_idx	scenario	model	params	cv_f1_macro_mean
0	1	S2_sans_variable_sensible	xgboost	{'colsample_bytree': 0.9, 'learning_rate': 0.0...	0.691366
1	5	S2_sans_variable_sensible	xgboost	{'colsample_bytree': 0.9, 'learning_rate': 0.0...	0.681618
2	2	S2_sans_variable_sensible	xgboost	{'colsample_bytree': 0.9, 'learning_rate': 0.0...	0.684514
3	6	S2_sans_variable_sensible	xgboost	{'colsample_bytree': 0.9, 'learning_rate': 0.0...	0.678675
4	4	S2_sans_variable_sensible	xgboost	{'colsample_bytree': 0.9, 'learning_rate': 0.0...	0.681085
5	3	S2_sans_variable_sensible	xgboost	{'colsample_bytree': 0.9, 'learning_rate': 0.0...	0.683066
6	7	S2_sans_variable_sensible	xgboost	{'colsample_bytree': 0.9, 'learning_rate': 0.0...	0.680275
7	8	S2_sans_variable_sensible	xgboost	{'colsample_bytree': 0.9, 'learning_rate': 0.0...	0.680716

```
Sélection finale section 5.5 (S2 production + tuning):
- scenario: S2_sans_variable_sensible
- model: xgboost
- params: {'colsample_bytree': 0.9, 'learning_rate': 0.03, 'max_depth': 5, 'n_est
imators': 300, 'subsample': 0.9}
- cv_recall_classe_2: 0.575
- cv_f1_macro: 0.691
- cv_roc_auc_ovr_macro: 0.857
```

5.6 Évaluation finale sur le test set

In [16]:
```
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score, classification_report, confusion_mat

assert "selected_scenario" in globals() and "selected_model_name" in globals(),
assert selected_scenario == "S2_sans_variable_sensible", "L'évaluation finale do

final_scenario = selected_scenario
final_model_name = selected_model_name
final_model = clone(candidate_models[final_model_name]).set_params(**selected_pa
final_label = "scénario S2 sans variable sensible, après tuning"
```

```
final_preprocessor = clone(scenario_data[final_scenario]["preprocessor"])
final_pipeline = Pipeline([
    ("prep", final_preprocessor),
    ("clf", final_model),
])

final_pipeline.fit(scenario_data[final_scenario]["X_train"], y_train)
y_pred = final_pipeline.predict(scenario_data[final_scenario]["X_test"])
y_proba = final_pipeline.predict_proba(scenario_data[final_scenario]["X_test"])

test_accuracy = accuracy_score(y_test, y_pred)
test_f1_macro = f1_score(y_test, y_pred, average="macro")
test_recall_class2 = recall_class_2(y_test, y_pred)
test_roc_auc_ovr_macro = roc_auc_score(y_test, y_proba, multi_class="ovr", avera

print("Résultats test set")
print(f"- variante: {final_label}")
print(f"- scenario: {final_scenario}")
print(f"- model: {final_model_name}")
print(f"- accuracy: {test_accuracy:.3f}")
print(f"- f1_macro: {test_f1_macro:.3f}")
print(f"- recall_classe_2: {test_recall_class2:.3f}")
print(f"- roc_auc_ovr_macro: {test_roc_auc_ovr_macro:.3f}")

print("\nClassification report:")
print(classification_report(y_test, y_pred, digits=3, zero_division=0))

cm = confusion_matrix(y_test, y_pred, labels=[0, 1, 2])
fig, ax = plt.subplots(figsize=(5, 4))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=[0, 1, 2])
disp.plot(ax=ax, colorbar=False)
ax.set_title("Matrice de confusion - modèle final S2")
plt.tight_layout()
plt.show()

section5_summary = {
    "variant": final_label,
    "best_scenario": final_scenario,
    "best_model": final_model_name,
    "test_accuracy": float(test_accuracy),
    "test_f1_macro": float(test_f1_macro),
    "test_recall_class2": float(test_recall_class2),
    "test_roc_auc_ovr_macro": float(test_roc_auc_ovr_macro),
}

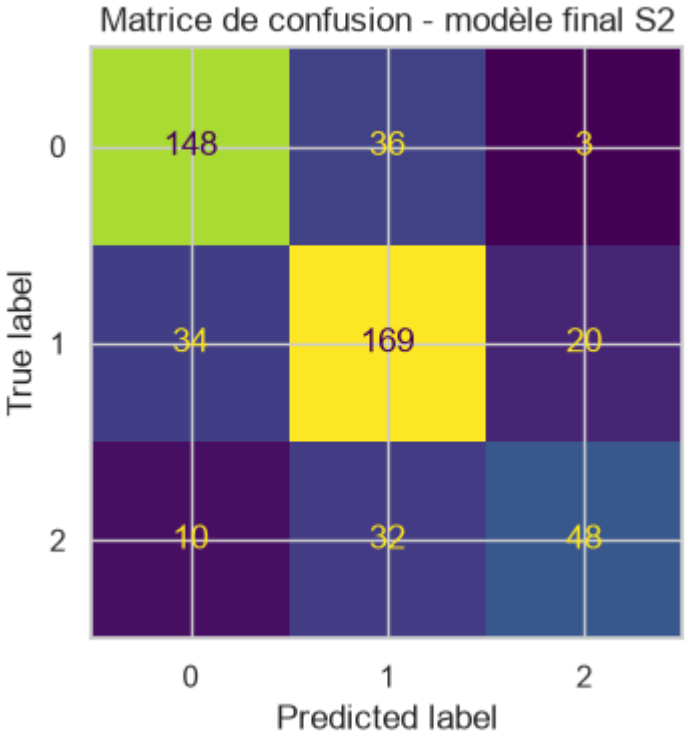
print("\nSynthèse section 5:", section5_summary)
```

```
Résultats test set
- variante: scénario S2 sans variable sensible, après tuning
- scenario: S2_sans_variable_sensible
- model: xgboost
- accuracy: 0.730
- f1_macro: 0.704
- recall_classe_2: 0.533
- roc_auc_ovr_macro: 0.871

Classification report:
              precision    recall  f1-score   support

           0      0.771     0.791     0.781       187
           1      0.713     0.758     0.735       223
           2      0.676     0.533     0.596        90

    accuracy                          0.730       500
   macro avg      0.720     0.694     0.704       500
weighted avg      0.728     0.730     0.727       500
```




```
Synthèse section 5: {'variant': 'scénario S2 sans variable sensible, après tunin
g', 'best_scenario': 'S2_sans_variable_sensible', 'best_model': 'xgboost', 'test_
accuracy': 0.73, 'test_f1_macro': 0.7040195130478484, 'test_recall_class2': 0.533
3333333333333, 'test_roc_auc_ovr_macro': 0.87061771666257}
```

5.7 Synthèse modélisation

Le meilleur score brut observé dans le protocole de comparaison est obtenu par **S1 + XGBoost**. Ce scénario est néanmoins conservé comme **référence expérimentale uniquement**, car il intègre directement `nationalite_hors_ue`, une variable sensible au sens du cadre réglementaire et métier.

Le modèle de production retenu est le meilleur modèle du scénario **S2 sans variable sensible**, après tuning et validation sur le jeu de test. Il conserve les apports du

multimodal (tabulaire + texte) tout en excluant explicitement la nationalité des features d'entraînement et d'inférence.

Le point de lecture important est le suivant : les scores peuvent varier légèrement selon le runtime XGBoost et la version de dépendances, mais la décision de conception reste stable. Le choix de S2 n'est donc pas un arbitrage purement de performance ; il est guidé par la conformité, la transparence et le contrôle des biais.

Les contrôles métier restent centrés sur le **F1 macro**, le **recall de la classe 2** et la fréquence des erreurs critiques `2 -> 0`. Les proxys à surveiller durant le monitoring restent `age`, `est_allocataire`, `departement_insee` et le texte libre.

---

6. Analyse des scénarios et arbitrages

Cette section compare les scénarios définis en section 4 selon un cadre commun : performance, risque de biais, coût et exploitabilité. L'objectif est de documenter une recommandation finale argumentée.

6.1 Tableau comparatif chiffré

Les chiffres ci-dessous utilisent le même protocole de split stratifié. Les métriques F1 et recall sont les moyennes de validation croisée ; les erreurs critiques sont comptées sur le jeu de test. La latence est un benchmark local reproductible, exprimé en p95 par prédiction. Le coût est un **proxy CPU** calculé avec l'hypothèse explicite de `0,05 EUR / vCPU-h` : il exclut réseau, stockage, haute disponibilité et coût fixe de l'infrastructure.

Scénario	Modèle de référence	F1 macro CV	Recall classe 2 CV	Erreurs 2 -> 0 sur test	Latence p95	Coût CPU estimé / 1 000 prédictions	Déc
S1 multimodal complet	XGBoost	**0,708**	**0,572**	**9**	**0,067 ms**	**0,000001 EUR**	Référence ex uniquement `nationali`
S2 sans variable sensible	XGBoost optimisé	**0,681**	**0,558**	**10**	**0,075 ms**	**0,000001 EUR**	**Production** compromis c performance
S3 texte seul	LogisticRegression	**0,635**	**0,661**	**12**	**0,020 ms**	**0,0000003 EUR**	Comparateu interprétable comme solu principale.
S4 tabulaire seul	RandomForest	**0,672**	**0,589**	**8**	**0,270 ms**	**0,000004 EUR**	Comparateu moins multi coûteux en i

Le **prix de la décision éthique** est mesurable : retirer `nationalite_hors_ue` fait passer le F1 macro de **0,708 à 0,681** (`-0,027`) et le recall de la classe 2 de **0,572 à**

0,558 (`-0,014`).

Les écarts de latence restent négligeables à l'échelle d'un entretien individuel. Le coût CPU est donc secondaire par rapport au coût métier d'une erreur critique et au coût de gouvernance d'une variable sensible.

6.2 Recommandation finale

Recommandation opérationnelle : retenir **S2 + XGBoost** pour la production.

La raison est une contrainte de conception, et non un simple arbitrage de score : `nationalite_hors_ue` est une variable sensible. Elle ne doit pas être conservée parmi les features d'entraînement ni transmise au modèle lors d'une inférence de production. Garder S1 comme référence est utile pour objectiver l'impact de cette exclusion ; ce n'est pas une justification pour l'utiliser en service.

S2 préserve le pipeline multimodal (données tabulaires + synthèse d'entretien) en retirant uniquement `nationalite_hors_ue` . Les métriques finales sont consignées dans les sorties des sections 5.4 à 5.6 et dans le tableau ci-dessus ; elles ne doivent pas être décrites comme des résultats « à réexécuter ».

Conditions de mise en oeuvre avant déploiement élargi :

- appliquer la politique d'escalade humaine calibrée hors test en section 7.2.1 ;
- suivre le recall classe 2, les erreurs critiques `2 -> 0` et le taux d'escalade ;
- auditer régulièrement les proxys potentiels (`age` , `est_allocataire` , `departement_insee` , texte libre) par sous-population ;
- réentraîner uniquement sur un jeu de données dont les variables sensibles directes sont exclues des features.

6.3 Synthèse arbitrages

Point de décision principal : **la conformité et l'absence de discrimination directe priment sur le gain de performance de S1**. S2 devient donc le scénario de référence pour l'industrialisation, avec une validation humaine maintenue sur les décisions incertaines ou sensibles.

Risques résiduels à traiter :

- biais indirects via des proxys et le texte libre ;
- recall classe 2 potentiellement inférieur à la cible métier ;
- latence p95 à mesurer dans un environnement API instrumenté.

Décision projet : industrialisation contrôlée de S2, avec un suivi continu de la performance, des erreurs critiques et des écarts entre sous-populations.

---

7. Interprétation et communication métier

Cette section transforme les résultats techniques en messages opérationnels compréhensibles pour un décideur non spécialiste. L'accent est mis sur les facteurs explicatifs, les erreurs résiduelles et les mécanismes de sécurité.

7.1 Variables explicatives du modèle final

Cette sous-section identifie les signaux les plus influents dans le modèle retenu. L'objectif n'est pas de fournir une causalité, mais une lecture opérationnelle des variables qui pèsent le plus dans la décision du classifieur.

In [17]:
```
import pandas as pd
import matplotlib.pyplot as plt

assert "final_pipeline" in globals(), "Exécuter la section 5.6 avant cette cellu

clf = final_pipeline.named_steps["clf"]
prep = final_pipeline.named_steps["prep"]

if not hasattr(clf, "feature_importances_"):
    raise TypeError("Le modèle final ne fournit pas de feature_importances_.")

num_names = [f"num__{name}" for name in final_preprocessor.transformers_[0][2]]
ord_names = [f"ord__{name}" for name in final_preprocessor.transformers_[1][2]]
cat_encoder = final_preprocessor.named_transformers_["cat"].named_steps["encoder
cat_input_names = final_preprocessor.transformers_[2][2]
cat_names = [f"cat__{name}" for name in cat_encoder.get_feature_names_out(cat_in
text_vectorizer = final_preprocessor.named_transformers_["txt"].named_steps["tfi
text_names = [f"txt__{name}" for name in text_vectorizer.get_feature_names_out()

feature_names = num_names + ord_names + cat_names + text_names
importance_df = pd.DataFrame({
    "feature": feature_names,
    "importance": clf.feature_importances_,
})

importance_df = importance_df.sort_values("importance", ascending=False).reset_i
top_features = importance_df.head(15).copy()


def feature_family(name: str) -> str:
    if name.startswith("txt__"):
        return "texte"
    if name.startswith("num__"):
        return "numerique"
    if name.startswith("ord__"):
        return "ordinale"
    if name.startswith("cat__"):
        return "categorielle"
    return "autre"


importance_df["family"] = importance_df["feature"].apply(feature_family)
family_importance = (
    importance_df.groupby("family", as_index=False)["importance"]
    .sum()
    .sort_values("importance", ascending=False)
    .reset_index(drop=True)
```

```
)

display(top_features)
display(family_importance)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].barh(top_features["feature"].iloc[::-1], top_features["importance"].iloc
axes[0].set_title("Top 15 features du modèle final")
axes[0].set_xlabel("importance")

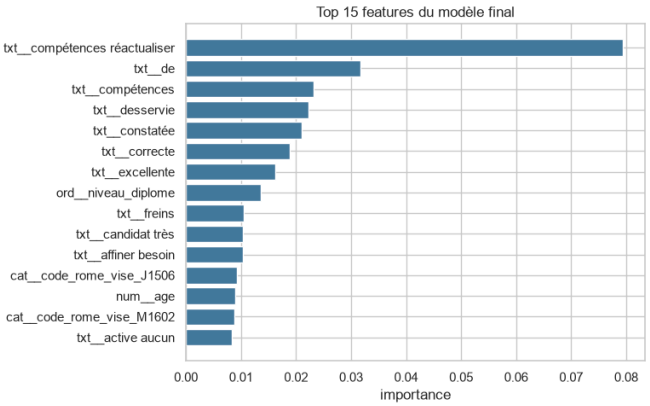
axes[1].bar(family_importance["family"], family_importance["importance"], color=
axes[1].set_title("Importance agrégée par famille de variables")
axes[1].set_ylabel("importance cumulée")

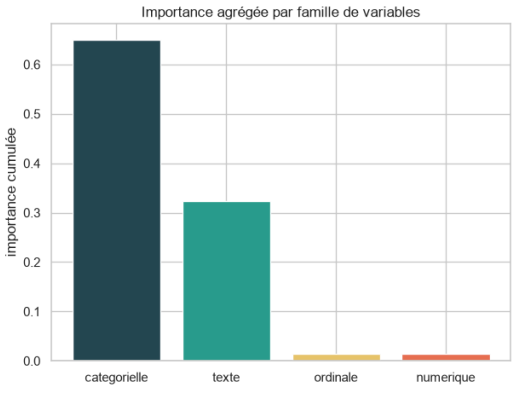
plt.tight_layout()
plt.show()

print("Lecture rapide section 7.1 :")
print(f"- famille dominante : {family_importance.iloc[0]['family']}")
print(f"- top feature : {top_features.iloc[0]['feature']}")
```

	feature	importance
0	txt__compétences réactualiser	0.079425
1	txt__de	0.031683
2	txt__compétences	0.023164
3	txt__desservie	0.022191
4	txt__constatée	0.020986
5	txt__correcte	0.018929
6	txt__excellente	0.016202
7	ord__niveau_diplome	0.013541
8	txt__freins	0.010439
9	txt__candidat très	0.010389
10	txt__affiner besoin	0.010374
11	cat__code_rome_vise_J1506	0.009226
12	num__age	0.008981
13	cat__code_rome_vise_M1602	0.008781
14	txt__active aucun	0.008313

	family	importance
0	categorielle	0.649372
1	texte	0.323559
2	ordinale	0.013541
3	numerique	0.013528






```
Lecture rapide section 7.1 :
- famille dominante : categorielle
- top feature : txt__compétences réactualiser
```

7.2 Analyse des erreurs et stratégies de fallback

Cette section formalise le comportement du système dans les cas incertains ou risqués, à partir du jeu de test utilisé pour la validation finale.

Avec la règle simple `confiance < 0,55`, le résultat observé est de **3 erreurs critiques captées sur 10** (`30 %`) pour **19,8 %** de dossiers escaladés. Le seuil `0,55` est donc un diagnostic de départ, pas la politique retenue : il ne couvre pas suffisamment le risque métier `2 -> 0`.

Levier	Question de conception	Décision retenue
Rejection threshold	À partir de quel niveau de confiance s'abstenir ?	Seuil `0,70`, calibré hors test sur des probabilités OOF du train.
Abstention contrôlée	Quelle réponse API renvoyer en cas d'abstention ?	Retourner `status = a_revoir`, avec classe proposée, score de confiance, version et `request_id`.
Escalade humaine (HITL)	Qui reprend la décision ?	Un conseiller ou référent métier reprend les dossiers escaladés avant toute orientation sensible.

La politique retenue est **combinée** : escalade si `confiance < 0,70` **ou** si le modèle prédit la classe 0 avec `P(classe 2) >= 0,20`. Elle répond directement à la consigne de modifier le meilleur modèle pour réduire les erreurs critiques : le modèle reste S2, mais sa règle de décision ajoute un garde-fou métier ciblé.

Le test du seuil `0,55` et le choix du seuil `0,70` sont séparés : le premier sert à montrer la limite de la règle naïve ; le second est choisi en section 7.2.1 hors test, puis vérifié une seule fois sur le test.

Le modèle reste un **outil d'aide à la décision**, non un automate de décision finale.

In [18]:
```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

assert "final_pipeline" in globals(), "Exécuter la section 5.6 avant cette cellu
```

```python
proba = final_pipeline.predict_proba(scenario_data[final_scenario]["X_test"])
confidence = proba.max(axis=1)
pred_class = proba.argmax(axis=1)

error_analysis = scenario_data[final_scenario]["X_test"].copy()
error_analysis["y_true"] = y_test.to_numpy()
error_analysis["y_pred"] = y_pred
error_analysis["confidence"] = confidence
error_analysis["is_error"] = error_analysis["y_true"] != error_analysis["y_pred"
error_analysis["is_critical_2_to_0"] = (error_analysis["y_true"] == 2) & (error_

threshold = 0.55
error_analysis["escalate_low_confidence"] = error_analysis["confidence"] < thres

summary_72 = {
    "n_test": int(len(error_analysis)),
    "error_rate": float(error_analysis["is_error"].mean()),
    "critical_2_to_0": int(error_analysis["is_critical_2_to_0"].sum()),
    "pct_low_confidence": float(error_analysis["escalate_low_confidence"].mean()
    "critical_captured_by_rule": int(error_analysis.loc[error_analysis["is_criti
}

profile_columns = [
    "y_true",
    "y_pred",
    "confidence",
    "is_error",
    "is_critical_2_to_0",
    "age",
    "niveau_diplome",
    "est_allocataire",
]
escalation_profiles = error_analysis.loc[
    error_analysis["escalate_low_confidence"] | error_analysis["is_critical_2_to
    [column for column in profile_columns if column in error_analysis.columns],
].head(15)

display(pd.DataFrame([summary_72]))
display(escalation_profiles)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].hist(error_analysis.loc[~error_analysis["is_error"], "confidence"], bins
axes[0].hist(error_analysis.loc[error_analysis["is_error"], "confidence"], bins=
axes[0].axvline(threshold, color="black", linestyle="--", label=f"seuil={thresho
axes[0].set_title("Confiance des prédictions : correctes vs erreurs")
axes[0].set_xlabel("probabilité max")
axes[0].legend()

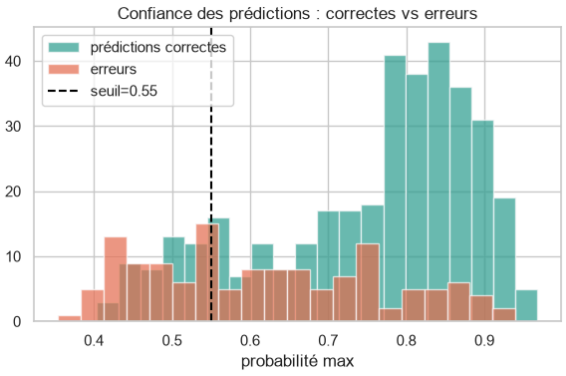
critical_counts = pd.Series({
    "2->0": int(error_analysis["is_critical_2_to_0"].sum()),
    "escaladés": int(error_analysis["escalate_low_confidence"].sum()),
    "2->0 escaladés": int(error_analysis.loc[error_analysis["is_critical_2_to_0"
})
axes[1].bar(critical_counts.index, critical_counts.values, color=["#d62828", "#5
axes[1].set_title("Couverture d'une règle simple d'escalade")
axes[1].set_ylabel("effectif")

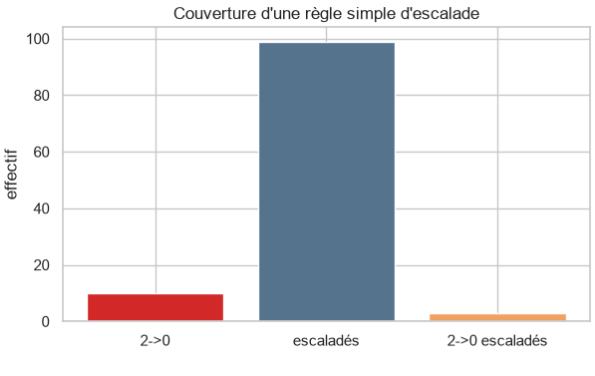
plt.tight_layout()
plt.show()
```

```
print("Lecture rapide section 7.2 :")
print(f"- seuil d'escalade testé : {threshold}")
print(f"- part de dossiers escaladés : {summary_72['pct_low_confidence']:.3f}")
print(f"- erreurs critiques 2->0 : {summary_72['critical_2_to_0']}")
print(f"- erreurs critiques captées par la règle : {summary_72['critical_capture
```

	n_test	error_rate	critical_2_to_0	pct_low_confidence	critical_captured_by_rule
0	500	0.27	10	0.198	3

	y_true	y_pred	confidence	is_error	is_critical_2_to_0	age	niveau_diplome	est_all
548	2	0	0.673850	True	True	34.0	Bac	
1186	1	0	0.520796	True	False	47.0	Bac+2	
1892	2	1	0.440465	True	False	24.0	Bac	
2267	1	2	0.531107	True	False	62.0	Bac	
1295	1	0	0.417130	True	False	60.0	Bac	
1850	2	0	0.651046	True	True	28.0	Bac	
1267	0	0	0.404837	False	False	22.0	Bac	
327	2	2	0.500904	False	False	61.0	Bac+5	
940	1	2	0.422530	True	False	NaN	Bac	
2470	0	0	0.485191	False	False	33.0	Bac+2	
778	2	1	0.499088	True	False	NaN	Bac	
1798	0	1	0.463940	True	False	58.0	Sans diplôme	
831	1	1	0.503164	False	False	58.0	Bac+2	
1995	1	2	0.429472	True	False	26.0	Sans diplôme	
105	1	1	0.460322	False	False	59.0	Bac+5	






```
Lecture rapide section 7.2 :
- seuil d'escalade testé : 0.55
- part de dossiers escaladés : 0.198
- erreurs critiques 2->0 : 10
- erreurs critiques captées par la règle : 3
```

7.2.1 Calibration hors échantillon et décision du seuil

Le seuil ne doit pas être choisi sur le jeu de test : cela transformerait le test en outil d'optimisation et rendrait l'évaluation trop optimiste.

La procédure est donc :

1. produire des probabilités **out-of-fold** sur le train avec `cross_val_predict(..., method="predict_proba")` ;
2. tester une grille de seuils et les politiques simple / combinée sur ces prédictions hors échantillon ;
3. retenir une règle avant de regarder le test ;
4. vérifier une seule fois la couverture et le taux d'escalade sur le test.

La décision projet est d'accepter un budget d'escalade proche de **40 %**. Sur les prédictions OOF, ce budget ne permet pas d'atteindre 80 % de couverture : nous relâchons donc la contrainte de couverture à **70 % minimum** pour le calibrage. Le seuil combiné `0,70` est retenu : il atteint environ **40,2 % d'escalade** et **72,5 % de couverture** sur le train OOF. La vérification finale sur le test donne **40,0 % d'escalade** et **8/10 erreurs critiques captées, soit 80 %**.

Cette décision est une hypothèse de capacité opérationnelle à valider avec l'agence : si 40 % de dossiers à revoir est trop élevé, il faudra accepter une couverture moindre ou financer davantage de reprise humaine.

In [19]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.base import clone
from sklearn.model_selection import cross_val_predict

assert "final_pipeline" in globals(), "Exécuter la section 5.6 avant cette cellu
assert "cv" in globals(), "La validation croisée de la section 5.2 doit être dis

# 1) Calibration uniquement sur le train avec des probabilités hors échantillon.
oof_pipeline = Pipeline([
    ("prep", clone(scenario_data[final_scenario]["preprocessor"])),
    ("clf", clone(final_model)),
])
oof_proba = cross_val_predict(
    oof_pipeline,
    scenario_data[final_scenario]["X_train"],
    y_train,
    cv=cv,
    method="predict_proba",
    n_jobs=1,
)
oof_confidence = oof_proba.max(axis=1)
oof_pred_class = oof_proba.argmax(axis=1)
oof_p_class2 = oof_proba[:, 2]
oof_y_true = y_train.to_numpy()
oof_critical = (oof_y_true == 2) & (oof_pred_class == 0)
oof_critical_total = int(oof_critical.sum())

threshold_grid = np.round(np.arange(0.40, 0.81, 0.05), 2)
risk_gate_class2 = 0.20
oof_rows = []
```

```python
for threshold in threshold_grid:
    policies = {
        "simple": oof_confidence < threshold,
        "combinee": (oof_confidence < threshold)
        | ((oof_pred_class == 0) & (oof_p_class2 >= risk_gate_class2)),
    }
    for policy_name, escalate_mask in policies.items():
        captured = int((oof_critical & escalate_mask).sum())
        oof_rows.append({
            "policy": policy_name,
            "threshold": float(threshold),
            "escalation_rate": float(escalate_mask.mean()),
            "critical_total": oof_critical_total,
            "critical_captured": captured,
            "critical_recall": captured / oof_critical_total if oof_critical_tot
        })

oof_sweep = pd.DataFrame(oof_rows)

# 2) Choix préalable au test : budget proche de 40 %, couverture OOF >= 70 %.
max_escalation_rate = 0.41
min_oof_critical_recall = 0.70
eligible = oof_sweep[
    (oof_sweep["policy"] == "combinee")
    & (oof_sweep["escalation_rate"] <= max_escalation_rate)
    & (oof_sweep["critical_recall"] >= min_oof_critical_recall)
]
if eligible.empty:
    raise RuntimeError("Aucun seuil combiné ne respecte les contraintes OOF rete

selected_policy = "combinee"
selected_threshold = float(
    eligible.sort_values(["escalation_rate", "critical_recall", "threshold"])
    .iloc[0]["threshold"]
)
oof_decision = eligible.loc[
    eligible["threshold"] == selected_threshold
].sort_values("critical_recall", ascending=False).iloc[0]

# 3) Vérification unique sur le test, après le choix du seuil.
test_proba = final_pipeline.predict_proba(scenario_data[final_scenario]["X_test"
test_confidence = test_proba.max(axis=1)
test_pred_class = test_proba.argmax(axis=1)
test_p_class2 = test_proba[:, 2]
test_y_true = y_test.to_numpy()
test_critical = (test_y_true == 2) & (test_pred_class == 0)
test_escalate = (test_confidence < selected_threshold) | (
    (test_pred_class == 0) & (test_p_class2 >= risk_gate_class2)
)
test_critical_total = int(test_critical.sum())
test_critical_captured = int((test_critical & test_escalate).sum())

test_decision = {
    "escalation_rate": float(test_escalate.mean()),
    "critical_total": test_critical_total,
    "critical_captured": test_critical_captured,
    "critical_recall": test_critical_captured / test_critical_total if test_crit
}

print("Décision de seuil 7.2.1")
```

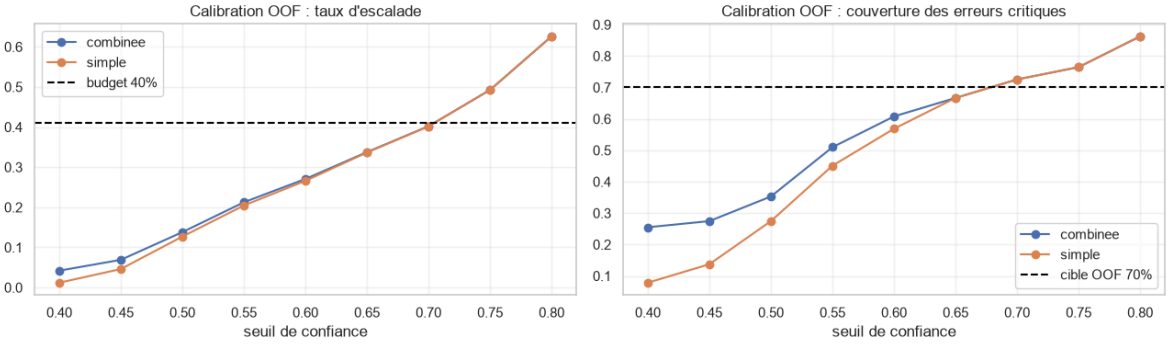
```
print(f"- politique retenue : {selected_policy}")
print(f"- seuil choisi sur OOF : {selected_threshold:.2f}")
print(f"- OOF : {oof_decision['escalation_rate']:.1%} d'escalade, {oof_decision[
print(f"- test de vérification : {test_decision['escalation_rate']:.1%} d'escala

display(oof_sweep.style.format({
    "threshold": "{:.2f}",
    "escalation_rate": "{:.1%}",
    "critical_recall": "{:.1%}",
}))

fig, axes = plt.subplots(1, 2, figsize=(13, 4))
for policy_name, group in oof_sweep.groupby("policy"):
    axes[0].plot(group["threshold"], group["escalation_rate"], marker="o", label
    axes[1].plot(group["threshold"], group["critical_recall"], marker="o", label
axes[0].axhline(max_escalation_rate, color="black", linestyle="--", label="budge
axes[1].axhline(min_oof_critical_recall, color="black", linestyle="--", label="c
axes[0].set_title("Calibration OOF : taux d'escalade")
axes[1].set_title("Calibration OOF : couverture des erreurs critiques")
for ax in axes:
    ax.set_xlabel("seuil de confiance")
    ax.grid(alpha=0.25)
    ax.legend()
plt.tight_layout()
plt.show()
```

```
Décision de seuil 7.2.1
- politique retenue : combinee
- seuil choisi sur OOF : 0.70
- OOF : 40.2% d'escalade, 72.5% de couverture
- test de vérification : 40.0% d'escalade, 8/10 erreurs critiques captées (80.0%)
```

	policy	threshold	escalation_rate	critical_total	critical_captured	critical_recall
0	simple	0.40	1.2%	51	4	7.8%
1	combinee	0.40	4.2%	51	13	25.5%
2	simple	0.45	4.6%	51	7	13.7%
3	combinee	0.45	6.9%	51	14	27.5%
4	simple	0.50	12.7%	51	14	27.5%
5	combinee	0.50	13.8%	51	18	35.3%
6	simple	0.55	20.4%	51	23	45.1%
7	combinee	0.55	21.2%	51	26	51.0%
8	simple	0.60	26.6%	51	29	56.9%
9	combinee	0.60	27.1%	51	31	60.8%
10	simple	0.65	33.7%	51	34	66.7%
11	combinee	0.65	33.8%	51	34	66.7%
12	simple	0.70	40.2%	51	37	72.5%
13	combinee	0.70	40.2%	51	37	72.5%
14	simple	0.75	49.2%	51	39	76.5%
15	combinee	0.75	49.2%	51	39	76.5%
16	simple	0.80	62.6%	51	44	86.3%
17	combinee	0.80	62.6%	51	44	86.3%



7.2.2 Audit d'équité de S2 et de la politique combinée

La nationalité est réutilisée uniquement pour l'audit, jamais comme feature S2. Les groupes proviennent des données sources, alignées sur les index du test. Les âges manquants ne sont pas imputés pour attribuer artificiellement une tranche.

Le rappel mesure la détection brute de classe 2 ; la couverture mesure le signalement des erreurs `2 -> 0`, pas leur correction effective par un conseiller. Ces indicateurs descriptifs ne certifient pas l'absence de discrimination.

In [20]:
```
from math import sqrt
from statistics import NormalDist
```

```
from IPython.display import display

assert final_scenario == "S2_sans_variable_sensible"
audit_features = scenario_data[final_scenario]["X_test"]
assert "nationalite_hors_ue" not in audit_features.columns
assert df.index.is_unique and audit_features.index.is_unique
assert audit_features.index.equals(y_test.index)
assert np.array_equal(final_pipeline.classes_, [0, 1, 2])
assert selected_policy == "combinee"


def wilson_interval(successes, total):
    if total == 0:
        return (np.nan, np.nan)
    critical_value = NormalDist().inv_cdf(0.975)
    proportion = successes / total
    denominator = 1 + critical_value ** 2 / total
    center = (proportion + critical_value ** 2 / (2 * total)) / denominator
    margin = critical_value * sqrt(
        proportion * (1 - proportion) / total + critical_value ** 2 / (4 * total
    ) / denominator
    return (max(0.0, center - margin), min(1.0, center + margin))


def interval_text(successes, total):
    lower, upper = wilson_interval(successes, total)
    return f"[{lower:.1%} ; {upper:.1%}]" if total else "Non estimable (aucun ca


audit_frame = df.loc[audit_features.index, ["nationalite_hors_ue", "age"]].copy(
assert df.loc[audit_features.index, target_col].equals(y_test)
audit_frame["y_true"] = y_test
assert test_proba.shape == (len(audit_frame), 3)
audit_frame["y_pred"] = final_pipeline.classes_[test_proba.argmax(axis=1)]
audit_frame["escalade"] = (test_proba.max(axis=1) < selected_threshold) | (
    (audit_frame["y_pred"].to_numpy() == 0) & (test_proba[:, 2] >= risk_gate_cla
)
assert np.array_equal(audit_frame["escalade"].to_numpy(), test_escalate)
audit_frame["classe_2"] = audit_frame["y_true"] == 2
audit_frame["classe_2_detectee"] = audit_frame["classe_2"] & (audit_frame["y_pre
audit_frame["critique_2_0"] = audit_frame["classe_2"] & (audit_frame["y_pred"] =
audit_frame["critique_non_escaladee"] = audit_frame["critique_2_0"] & ~audit_fra
audit_frame["groupe_nationalite"] = (
    audit_frame["nationalite_hors_ue"].map({0: "UE", 1: "Hors UE"}).fillna("Non
)
audit_frame["tranche_age"] = pd.cut(
    audit_frame["age"], bins=[-np.inf, 30, 50, np.inf], right=False,
    labels=["Moins de 30 ans", "30 a 49 ans", "50 ans et plus"],
).astype("string").fillna("Age manquant")

rows_audit = []
for dimension, group_column in [("Nationalite", "groupe_nationalite"), ("Age", "
    for group_name, group in audit_frame.groupby(group_column, sort=True):
        group_size = len(group)
        risk_count = int(group["classe_2"].sum())
        detected_count = int(group["classe_2_detectee"].sum())
        critical_count = int(group["critique_2_0"].sum())
        residual_count = int(group["critique_non_escaladee"].sum())
        escalation_count = int(group["escalade"].sum())
        rows_audit.append({
```

```
            "dimension": dimension,
            "groupe": group_name,
            "effectif": group_size,
            "nombre_classe_2": risk_count,
            "recall_classe_2": detected_count / risk_count if risk_count else np
            "IC95_recall": interval_text(detected_count, risk_count),
            "nombre_2_0": critical_count,
            "nombre_2_0_non_escalades": residual_count,
            "couverture_2_0": (critical_count - residual_count) / critical_count
            "IC95_couverture": interval_text(critical_count - residual_count, cr
            "nombre_escalades": escalation_count,
            "taux_escalade": escalation_count / group_size,
        })

equity_audit = pd.DataFrame(rows_audit)
for dimension in ["Nationalite", "Age"]:
    subset = equity_audit.loc[equity_audit["dimension"] == dimension]
    assert int(subset["effectif"].sum()) == len(audit_frame)
    assert int(subset["nombre_classe_2"].sum()) == int(audit_frame["classe_2"].s
    assert int(subset["nombre_2_0"].sum()) == test_critical_total
    assert int(subset["nombre_2_0_non_escalades"].sum()) == test_critical_total
assert np.isnan(wilson_interval(0, 0)).all()
assert abs(wilson_interval(0, 10)[0]) < 1e-12
assert abs(wilson_interval(10, 10)[1] - 1.0) < 1e-12
assert np.allclose(wilson_interval(8, 10), [0.49016247153664183, 0.9433178485456

display(equity_audit.style.format({
    "recall_classe_2": "{:.1%}", "couverture_2_0": "{:.1%}", "taux_escalade": "{
}, na_rep="Non estimable"))
print(equity_audit.to_string(index=False))

system_sample_size = len(audit_frame)
system_escalations = int(audit_frame["escalade"].sum())
system_residual_errors = int(audit_frame["critique_non_escaladee"].sum())
system_uncertainty = {
    "escalade_IC95": interval_text(system_escalations, system_sample_size),
    "couverture_IC95": interval_text(test_critical_captured, test_critical_total
    "risque_residuel_IC95": interval_text(system_residual_errors, system_sample_
}
print("Bilan client :", {
    "dossiers_test": system_sample_size, "dossiers_signales_pour_revue": system_
    "erreurs_critiques": test_critical_total, "erreurs_critiques_signalees": tes
    "erreurs_critiques_non_escaladees": system_residual_errors, **system_uncerta
})
```

	dimension	groupe	effectif	nombre_classe_2	recall_classe_2	IC95_recall	nombre_2_
0	Nationalite	Hors UE	56	20	45.0%	[25.8% ; 65.8%]	
1	Nationalite	UE	444	70	55.7%	[44.1% ; 66.8%]	
2	Age	30 a 49 ans	190	11	27.3%	[9.7% ; 56.6%]	
3	Age	50 ans et plus	158	57	73.7%	[61.0% ; 83.4%]	
4	Age	Age manquant	22	5	20.0%	[3.6% ; 62.4%]	
5	Age	Moins de 30 ans	130	17	11.8%	[3.3% ; 34.3%]	

```
  dimension          groupe  effectif  nombre_classe_2  recall_classe_2     IC95_
recall  nombre_2_0  nombre_2_0_non_escalades  couverture_2_0  IC95_couverture  no
mbre_escalades  taux_escalade
Nationalite         Hors UE        56               20         0.450000 [25.8% ;
65.8%]           2                          0            1.00 [34.2% ; 100.0%]
32       0.571429
Nationalite              UE       444               70         0.557143 [44.1% ;
66.8%]           8                          2            0.75  [40.9% ; 92.9%]
168       0.378378
        Age     30 a 49 ans       190               11         0.272727  [9.7% ;
56.6%]           3                          0            1.00 [43.9% ; 100.0%]
61       0.321053
        Age  50 ans et plus       158               57         0.736842 [61.0% ;
83.4%]           1                          1            0.00   [0.0% ; 79.3%]
66       0.417722
        Age    Age manquant        22                5         0.200000  [3.6% ;
62.4%]           1                          0            1.00 [20.7% ; 100.0%]
12       0.545455
        Age Moins de 30 ans       130               17         0.117647  [3.3% ;
34.3%]           5                          1            0.80  [37.6% ; 96.4%]
61       0.469231
Bilan client : {'dossiers_test': 500, 'dossiers_signales_pour_revue': 200, 'erreu
rs_critiques': 10, 'erreurs_critiques_signalees': 8, 'erreurs_critiques_non_escal
adees': 2, 'escalade_IC95': '[35.8% ; 44.4%]', 'couverture_IC95': '[49.0% ; 94.
3%]', 'risque_residuel_IC95': '[0.1% ; 1.4%]'}
```

Cadre de l'audit et usage distinct de la nationalité

La nationalité est exclue des features S2 et du payload d'inférence. Elle est reprise **uniquement depuis les données sources du test pour mesurer les écarts entre groupes**, sans modifier la prédiction ni choisir un seuil par nationalité. Les âges utilisés sont ceux déclarés avant imputation ; les 22 âges manquants restent dans une catégorie séparée.

Cet usage d'audit doit être encadré avec le DPO : finalité explicite, base légale appropriée, nécessité et proportionnalité documentées, accès restreints, durée de conservation définie et résultats agrégés. Cela n'autorise pas à réintroduire la nationalité dans `/predict`, dans l'UI ou dans les logs de production. Un audit futur supposerait

une jointure sécurisée et autorisée dans un espace séparé ; il n'est pas déjà automatisé par Grafana.

Les intervalles à 95 % sont des **intervalles binomiaux de Wilson**, calculés sur le bon dénominateur : personnes de classe 2 pour le rappel, erreurs `2 -> 0` pour la couverture et dossiers pour l'escalade. Ils décrivent l'incertitude d'échantillonnage sous une hypothèse de dossiers indépendants ; ils ne couvrent ni la dérive future, ni l'incertitude du choix des hyperparamètres, ni la réussite d'une relecture humaine. L'audit est descriptif : aucun seuil n'est retouché à partir de ces résultats sur le test déjà utilisé dans les analyses antérieures.

Conclusion de l'audit en cinq lignes

1. Le test contient **56 dossiers hors UE, dont 20 de classe 2**, contre **444 UE, dont 70 de classe 2** : le rappel brut est **45,0 % (9/20)** contre **55,7 % (39/70)** ; on ne peut pas affirmer une détection égale.
2. Le filet signale **2/2 erreurs critiques hors UE** et **6/8 UE** : aucune erreur `2 -> 0` hors UE ne reste non escaladée dans cet échantillon, contre **2 UE**, au prix d'une escalade de **57,1 % contre 37,8 %**.
3. Les groupes d'âge comptent **130 dossiers <30 ans (17 classe 2), 190 de 30 à 49 ans (11), 158 de 50 ans et plus (57)** : les rappels sont **11,8 %, 27,3 % et 73,7 %**, avec respectivement **1, 0 et 1 erreurs critiques non escaladées**.
4. Les **22 âges manquants (5 classe 2)** ont un rappel de **20,0 %** et une escalade de **54,5 %** ; les petits dénominateurs rendent les conclusions fragiles, notamment le **2/2 hors UE, IC95 ≈ 34–100 %**.
5. En production, suivre par groupe les effectifs étiquetés, le rappel classe 2, les erreurs `2 -> 0` non escaladées et la charge d'escalade avec leurs intervalles ; priorité aux jeunes, aux âges manquants et à la charge hors UE, sans prétendre que l'exclusion garantit l'équité.

7.3 Message au client (langage métier)

Le modèle S2 combine les informations socio-professionnelles et la synthèse d'entretien, sans utiliser la nationalité pour prédire. Le diplôme suit un ordre explicite (`Sans diplôme < Bac < Bac+2 < Bac+5`) ; cette hiérarchie ne signifie pas une relation causale avec le retour à l'emploi.

Avec la politique combinée évaluée, sur les **500 dossiers de test**, **200 sont signalés pour relecture par un conseiller**, soit **40 %**. Parmi les **10 personnes à risque de longue durée que le modèle classe à tort en retour rapide**, **8 sont signalées pour revue (80 %)** et **2 passent sans relecture déclenchée par cette règle**. Cela représente **0,4 % de tous les dossiers de test**. Le dispositif repère des erreurs potentielles ; leur correction effective dépend du conseiller et n'a pas été mesurée.

Ces résultats sont incertains : les intervalles de confiance binomiaux de Wilson à **95 %** sont environ **35,8–44,4 % pour le taux de relecture**, **49,0–94,3 % pour la couverture des erreurs critiques** et **0,1–1,4 % pour le taux d'erreurs critiques non escaladées sur**

l'ensemble des dossiers. La fourchette de couverture est particulièrement large car elle repose sur seulement **10 erreurs**. Ce sont des estimations sur cet échantillon, pas une garantie de performance future ; 80 % de couverture ne signifie pas 80 % de rappel de classe 2.

L'audit révèle aussi des écarts entre groupes : le filet capture les **2 erreurs critiques observées hors UE**, mais leur faible nombre ne démontre pas une équité générale. Les jeunes ont un rappel brut nettement inférieur à celui des plus âgés. Une vigilance par groupe est donc nécessaire, même sans nationalité dans les features.

Décision et périmètre : accepter un budget de reprise proche de **40 %** (borne OOF **41 %**) afin d'atteindre au moins **70 % de couverture OOF**, sous réserve de capacité opérationnelle de l'agence. Cette politique est évaluée et recommandée dans le notebook ; **l'API actuelle applique encore le seuil simple 0,55**, sans risk-gate combiné. Les chiffres ci-dessus ne décrivent donc pas une performance déjà déployée sur le terrain.

7.4 Synthèse interprétation

Points clés à retenir :

- S2 exclut `nationalite_hors_ue` de la prédiction ; son usage distinct pour l'audit doit être encadré côté RGPD.
- Les importances sont lues par familles : numérique, ordinale, catégorielle et texte ; elles ne prouvent pas une causalité.
- La politique combinée signale davantage d'erreurs critiques, avec une charge de reprise humaine explicite et des erreurs résiduelles.
- Les effectifs, les dénominateurs et les intervalles par groupe sont nécessaires pour interpréter les écarts d'équité.

Recommandations opérationnelles :

- conserver le modèle comme aide à la décision et non comme décisionnaire autonome ;
- valider la capacité de relecture et distinguer signalement automatique et correction humaine ;
- suivre les rappels, les erreurs résiduelles et les escalades par groupe sur les retours effectivement étiquetés ;
- recalibrer hors test après un réentraînement, puis vérifier sur des données indépendantes, sans optimiser les seuils à partir de l'audit du test.

---

8. Industrialisation

Cette section décrit l'architecture de mise en production: persistance du modèle, API, conteneurisation et chaîne d'intégration continue. Le code applicatif vit dans le repository; le notebook conserve la vue d'ensemble et les choix techniques.

Composants visés:

- API FastAPI (`/predict` , `/health` , `/retrain` , `/history` , `/metrics`);
- validation stricte des entrées et refus de la variable sensible `nationalite_hors_ue` ;
- journalisation structurée;
- tests automatisés;
- conteneurisation Docker;
- CI/CD;
- suivi d'expériences (MLflow).

8.1 Architecture SI implémentée (état actuel)

L'industrialisation est effective dans le repository avec une architecture locale complète:

- API FastAPI pour l'inférence, la santé du service, le retrain, le monitoring des modèles et la collecte de feedback ;
- artefact XGBoost du scénario S2, sans `nationalite_hors_ue` ;
- `niveau_diplome` encodé ordinalement (`Sans diplôme < Bac < Bac+2 < Bac+5`) ;
- UI statique servie par l'API (`/`) ;
- supervision Prometheus + Grafana via Docker Compose ;
- suivi d'expériences MLflow pour les runs d'entraînement ;
- CI/CD GitHub Actions (qualité, tests, entraînement de vérification, build d'image, publication).

8.2 Contrat API

Endpoints exposés :

- `GET /` : page UI de démonstration ;
- `GET /health` : statut de santé de l'API ;
- `POST /predict` : inférence sur un dossier ;
- `GET /metrics` : métriques Prometheus ;
- `POST /retrain` : cycle de réentraînement et décision de promotion (`promoted` / `rejected` / `skipped`) sur le scénario S2 ;
- `GET /models/history` : historique des snapshots de modèles ;
- `POST /rollback` : restauration d'une version précédente ;
- `POST /feedback` , `GET /feedback/count` , `GET /feedback/health` : collecte et pilotage des annotations terrain.

Note de conception importante :

- `usager_id` peut être transmis pour la traçabilité applicative ; il est exclu de la modélisation ;
- `nationalite_hors_ue` est interdite par le contrat API et retirée du pipeline de production ;

- `niveau_diplome` est encodé selon un ordre métier explicite ;
- les catégories nominales (`code_rome_vise` , `est_allocataire` , `departement_insee`) utilisent un one-hot encoding.

Exemple de payload `POST /predict` (minimal recommandé) :

```
{
  "age": 41,
  "niveau_diplome": "Bac+2",
  "anciennete_poste_ans": 6.5,
  "code_rome_vise": "M1602",
  "est_allocataire": 1,
  "departement_insee": "75",
  "synthese_entretien": "Recherche active, besoin de remise à niveau
numérique."
}
```

Exemple de réponse `POST /predict` :

```
{
  "prediction": 1,
  "confidence": 0.78,
  "status": "ok",
  "model_version": "xgb-s2-2026-09-01T12:15:34.001374+00:00",
  "request_id": "req_20260901_121534_ab12cd"
}
```

8.3 Modèle, artefacts et retrain

Artefacts persistés :

- `models/best_model/model.joblib` ;
- `models/best_model/metadata.json` (version, métriques, scénario S2, contexte d'entraînement et infos MLflow) ;
- `models/best_model/history/*.joblib` et `metadata_*.json` pour le rollback et l'historique des modèles.

Comportement de retrain :

1. L'API reçoit `POST /retrain` (ou le cron `retrain-cron` toutes les 6h).
2. `run_retrain_cycle(...)` (`scripts/retrain_feedback.py`) entraîne un candidat sur l'historique enrichi des feedbacks joints, avec le scénario S2 sans `nationalite_hors_ue` .
3. Le candidat est évalué face à la production sur le `reference_set` figé ; `decide_promotion()` tranche (recall classe 2 prioritaire, cf. section 9.3).
4. Les artefacts ne sont réécrits dans `models/best_model` **que si le candidat est promu** ; un rejet ne modifie rien.
5. En cas de promotion, le modèle courant est archivés avant remplacement; il reste disponible via `/models/history` et `/rollback` .
6. Chaque cycle (promu ou rejeté) est journalisé : événement JSONL et run MLflow dédié.

8.4 Observabilité implémentée

Supervision en place :

- Prometheus scrape `api:8000/metrics` ;
- Grafana provisionné automatiquement (datasource + dashboard) ;
- métriques API (compteurs/latences predict et retrain) ;
- métriques dérivées JSONL (volume d'événements predict/retrain, erreurs de retrain) ;
- métriques MLflow dérivées (`experiments_total` , `runs_total`).

Logs structurés :

- `logs/inference/api_predict_requests.jsonl` ;
- `logs/inference/api_train_events.jsonl` .

8.5 Conteneurisation et exécution locale

Éléments Docker :

- `infra/docker/Dockerfile` construit l'API et embarque `app/ui` ;
- `docker-compose.yml` orchestre API, Prometheus et Grafana ;
- volumes persistants pour les artefacts modèle, les logs et `mlruns` .

Conséquence pratique : l'UI et la supervision sont disponibles en environnement conteneurisé sans adaptation de code.

8.6 CI/CD (version courante)

Pipeline GitHub Actions actif :

- checks qualité ;
- tests ;
- entraînement S2 de vérification ;
- build image Docker ;
- publication GHCR ;
- hook de déploiement conditionnel (si secret configuré).

L'objectif de cette chaîne est d'assurer la cohérence du code, la reproductibilité du pipeline et la présence de l'image de production ; elle ne remplace pas les garde-fous métier du cycle de retrain, notamment la politique de décision sur le recall classe 2.

8.7 Garde-fous et conformité (niveau projet)

Principes appliqués :

- séparation identifiants techniques vs features modèle (exclusion `usager_id`) ;
- exclusion de `nationalite_hors_ue` du modèle et du contrat API ;
- encodage ordinal explicite de `niveau_diplome` ;
- journalisation des inférences et entraînements pour audit ;
- modèle utilisé comme aide à la décision (statut `a_revoir` si faible confiance).

8.8 Synthèse industrialisation

La chaîne MLOps minimale est opérationnelle de bout en bout : entraînement S2, service d'inférence, historique/rollback, monitoring, conteneurisation et CI.

La prochaine marche de maturité porte surtout sur :

- durcissement sécurité (authN/authZ API, gestion des secrets, TLS bout-en-bout) ;
- automatisation plus stricte des tests de non-régression métier ;
- enrichissement de la boucle de feedback terrain pour améliorer la détection de la classe 2 ;
- surveillance explicite des seuils de promotion et de calibration en production.

8.9 Dossier de livraison et preuves d'industrialisation

Cette sous-section rassemble les éléments demandés pour la livraison : code, configuration, dépôt et preuves visuelles. Les fichiers complets restent versionnés dans le repository.

Repository : github.com/userZku/multimodal-classification

Code API FastAPI

Fichier complet : `src/api/main.py`

```
@app.post("/predict", response_model=PredictResponse)
def predict(payload: PredictRequest) -> PredictResponse:
    frame = pd.DataFrame([payload.model_dump()])
    model_frame = build_model_frame(frame)
    proba = PIPELINE.predict_proba(model_frame)[0]
    prediction = int(proba.argmax())
    confidence = float(proba.max())
    status = "a_revoir" if confidence < ABSTENTION_THRESHOLD else "ok"
    return PredictResponse(
        prediction=prediction,
        confidence=confidence,
        status=status,
        model_version=model_version(),
        request_id=request_id,
    )
```

Les endpoints de cycle de vie sont également implémentés dans ce fichier : `/health`, `/retrain`, `/models/history`, `/rollback`, `/feedback`, `/feedback/count`, `/feedback/health` et `/metrics`.

Code UI

Fichiers complets : `app/ui/index.html`, `app/ui/app.js`, `app/ui/styles.css`

```
<form id="predict-form" class="predict-form">
  <input name="age" type="number" value="36" />
```

```
  <select name="niveau_diplome">
    <option value="Bac+2" selected>Bac+2</option>
  </select>
  <textarea name="synthese_entretien">Motivation stable, projet
cohérent, recherche active.</textarea>
  <button type="submit">Lancer la prédiction</button>
</form>
const response = await fetch("/predict", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(payload),
});
const result = await response.json();
renderPrediction(result);
```

Dockerfile

Fichier complet : `infra/docker/Dockerfile`

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt
COPY src ./src
COPY app ./app
COPY scripts ./scripts
COPY data ./data
RUN mkdir -p models logs
EXPOSE 8000
CMD ["python", "-m", "uvicorn", "src.api.main:app", "--host",
"0.0.0.0", "--port", "8000"]
```

Workflows CI/CD

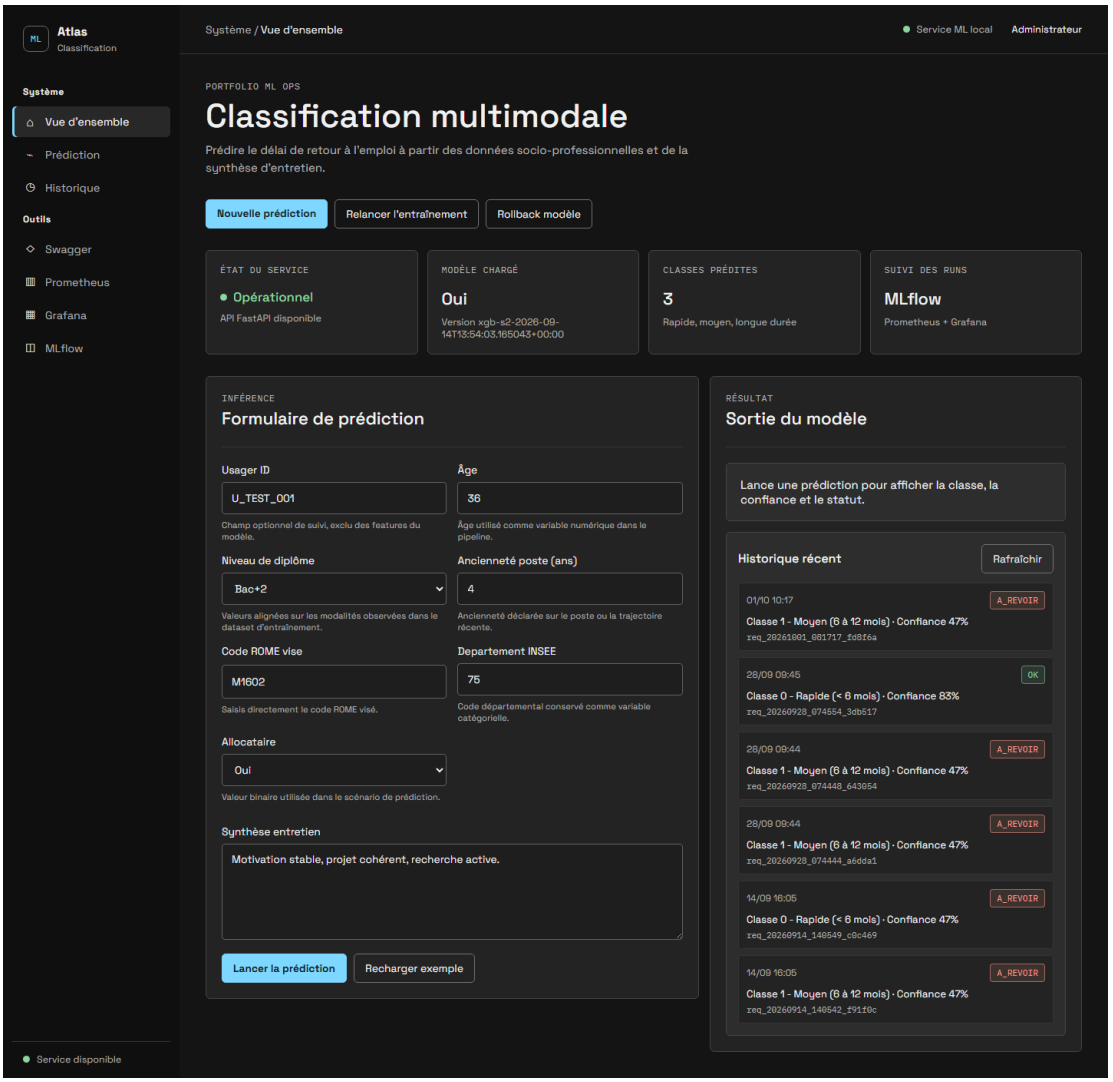
Fichiers complets : `.github/workflows/ci.yml` et `.github/workflows/retrain.yml`

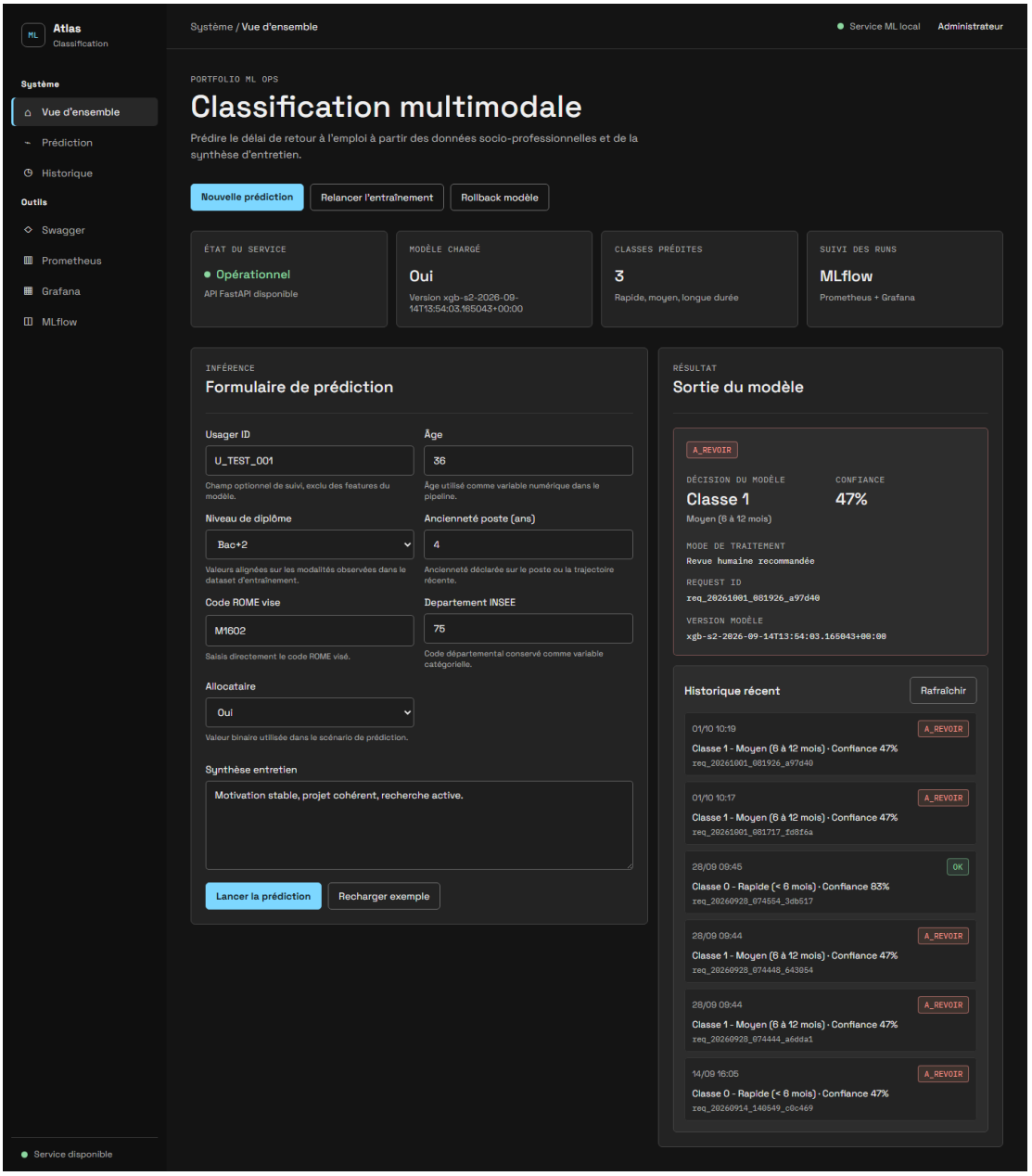
```
# ci.yml : qualité, tests, entraînement smoke, build et publication
GHCR
jobs:
  quality:
    steps: [checkout, setup-python, install-dependencies, pytest]
  training_smoke:
    needs: [quality]
  docker_build:
    needs: [prepare, quality, training_smoke]
  docker_publish:
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
# retrain.yml : déclenchement manuel ou quotidien
on:
  workflow_dispatch:
  schedule:
    - cron: "0 3 * * *"

# Le candidat est évalué, puis le déploiement est conditionné à la
```

```
promotion.
python scripts/retrain_feedback.py --min-feedback "$MIN_FB"
```

Captures de démonstration




Atlas
Classification
Système
Vue d'ensemble
Prédiction
Historique
Outils
Swagger
Prometheus
Grafana
MLflow
Service disponible
Système / Vue d'ensemble
Service ML local
Administrateur
PORTFOLIO ML OPS
Classification multimodale
Prédire le délai de retour à l'emploi à partir des données socio-professionnelles et de la synthèse d'entretien.
Nouvelle prédiction
Relancer l'entraînement
Rollback modèle
ÉTAT DU SERVICE
Opérationnel
API FastAPI disponible
MODÈLE CHARGÉ
Oui
Version xgb-s2-2026-09-14T13:54:03.165043+00:00
CLASSES PRÉDITES
3
Rapide, moyen, longue durée
SUIVI DES RUNS
MLflow
Prometheus + Grafana
INFÉRENCE
Formulaire de prédiction
Usager ID
U_TEST_001
Champ optionnel de suivi, exclu des features du modèle.
Âge
36
Âge utilisé comme variable numérique dans le pipeline.
Niveau de diplôme
Bac+2
Valeurs alignées sur les modalités observées dans le dataset d'entraînement.
Ancienneté poste (ans)
4
Ancienneté déclarée sur le poste ou la trajectoire récente.
Code ROME visé
M1602
Saisis directement le code ROME visé.
Département INSEE
75
Code départemental conservé comme variable catégorielle.
Allocataire
Oui
Valeur binaire utilisée dans le scénario de prédiction.
Synthèse entretien
Motivation stable, projet cohérent, recherche active.
Lancer la prédiction
Recharger exemple
RÉSULTAT
Sortie du modèle
A_REVOIR
DÉCISION DU MODÈLE
Classe 1
Moyen (6 à 12 mois)
CONFIANCE
47%
MODE DE TRAITEMENT
Revue humaine recommandée
REQUEST ID
req_20261001_081926_a97d40
VERSION MODÈLE
xgb-s2-2026-09-14T13:54:03.165043+00:00
Historique récent
Rafraîchir
01/10 10:19
A_REVOIR
Classe 1 - Moyen (6 à 12 mois) · Confiance 47%
req_20261001_081926_a97d40
01/10 10:17
A_REVOIR
Classe 1 - Moyen (6 à 12 mois) · Confiance 47%
req_20261001_081717_fd8f6a
28/09 09:45
OK
Classe 0 - Rapide (< 6 mois) · Confiance 83%
req_20260928_074554_3db517
28/09 09:44
A_REVOIR
Classe 1 - Moyen (6 à 12 mois) · Confiance 47%
req_20260928_074448_643054
28/09 09:44
A_REVOIR
Classe 1 - Moyen (6 à 12 mois) · Confiance 47%
req_20260928_074444_a6dda1
14/09 16:05
A_REVOIR
Classe 0 - Rapide (< 6 mois) · Confiance 47%
req_20260914_140549_c0c469

Multimodal Classification API

1.0.0 OAS 3.1

/openapi.json

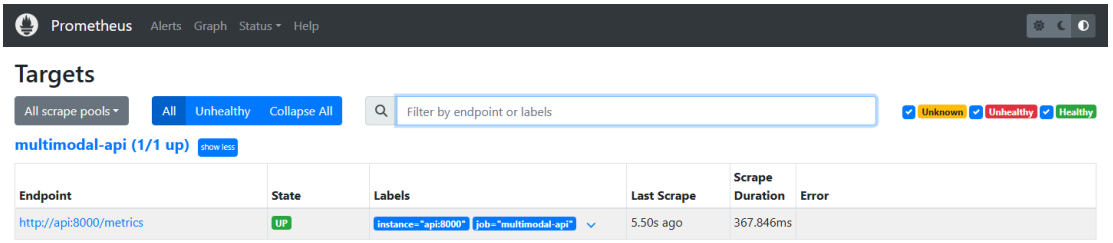
Inference API for return-to-employment delay classification.

default

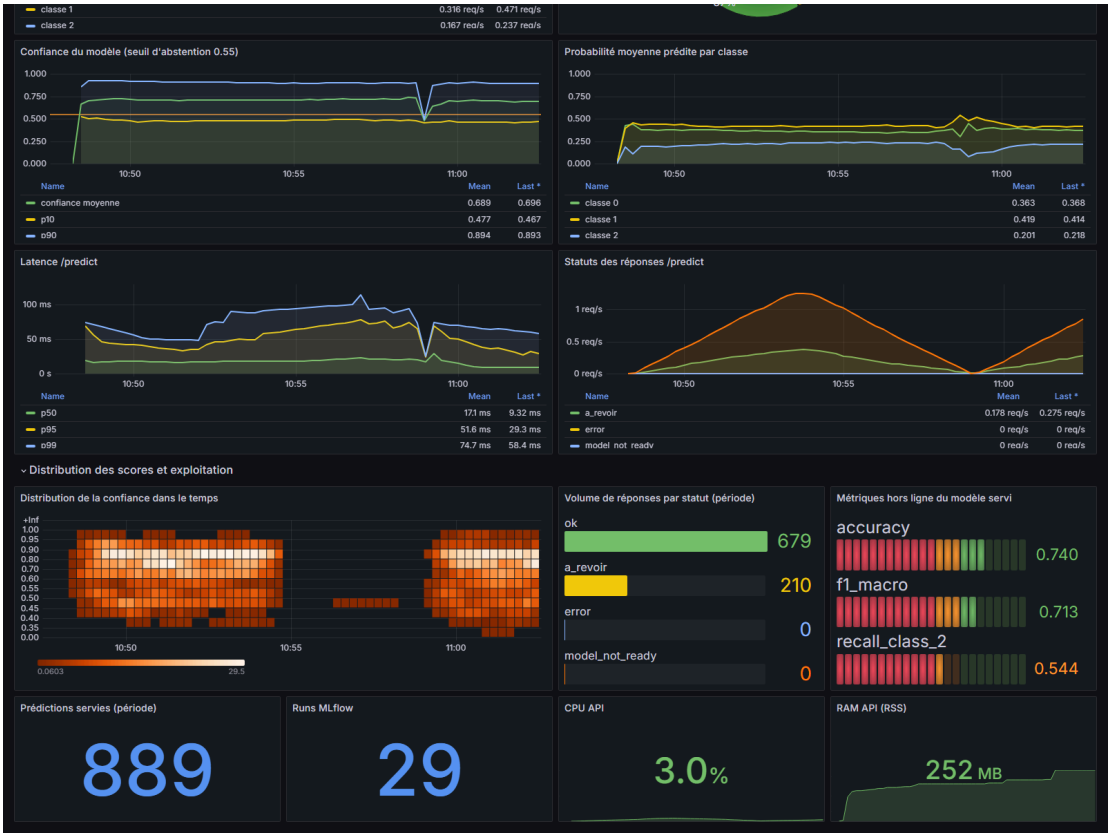
- GET /health Health
- GET /metrics Metrics
- GET /history History
- POST /predict Predict
- POST /retrain Retrain
- GET /models/history Models History
- POST /rollback Rollback
- POST /feedback Post Feedback
- GET /feedback/count Get Feedback Count
- GET /feedback/health Feedback Health

Schemas

- FeedbackCountResponse › Expand all object
- FeedbackRequest › Expand all object
- FeedbackResponse › Expand all object
- HTTPValidationError › Expand all object
- HealthResponse › Expand all object
- ModelHistoryEntry › Expand all object
- ModelHistoryResponse › Expand all object
- PredictRequest › Expand all object
- PredictResponse › Expand all object
- RollbackRequest › Expand all object
- RollbackResponse › Expand all object
- TrainRequest › Expand all object
- TrainResponse › Expand all object
- ValidationError › Expand all object


Prometheus
Alerts
Graph
Status
Help
Targets
All scrape pools
All
Unhealthy
Collapse All
Filter by endpoint or labels
Unknown
Unhealthy
Healthy
multimodal-api (1/1 up)
show less
Endpoint
State
Labels
Last Scrape
Scrape Duration
Error
http://api:8000/metrics
UP
instance="api:8000"
job="multimodal-api"
5.50s ago
367.846ms


Service et qualité du modèle
API
UP
Débit prédictions (5m)
1.13 req/s
Taux d'erreur /predict (5m)
0.0%
Taux d'abstention (a_revoir)
24.4%
Latence p95
29 ms
F1 macro (modèle actif)
0.713
Réponses du modèle
Prédictions par classe (débit 5m)
1.5 req/s
1 req/s
0.5 req/s
0 req/s
10:50
10:55
11:00
Name
Mean
Last *
classe 0 0.282 req/s 0.417 req/s
classe 1 0.316 req/s 0.471 req/s
classe 2 0.167 req/s 0.237 req/s
Répartition des classes prédites (période affichée)
Value
Percent
classe 1 366 41%
classe 0 329 37%
classe 2 194 22%
22%
41%
37%
Confiance du modèle (seuil d'abstention 0.55)
1.000
0.750
0.500
0.250
0.000
confiance moyenne 0.689 0.696
p10 0.477 0.467
p90 0.894 0.893
Probabilité moyenne prédite par classe
classe 0 0.363 0.368
classe 1 0.419 0.414
classe 2 0.201 0.218
Latence /predict
100 ms
50 ms
0 s
p50 17.1 ms 9.32 ms
p95 51.6 ms 29.3 ms
p99 74.7 ms 58.4 ms
Statuts des réponses /predict
1 req/s
0.5 req/s
0 req/s
a_revoir 0.178 req/s 0.275 req/s
error 0 req/s 0 req/s
model not readv 0 req/s 0 req/s



Les preuves complémentaires sont accessibles directement via les services locaux : Swagger (`http://localhost:8000/docs`), Grafana (`http://localhost:3000`) et MLflow (`http://localhost:5000`). Le dépôt ne contient pas de capture dédiée du run MLflow ni du run CI vert ; la preuve CI reste consultable dans l'onglet Actions du repository, et les workflows sont reproduits ci-dessus.

8.10 Preuves de build Docker et de CI/CD

Build Docker local

Commande exécutée depuis la racine du dépôt le **04/10/2026** :

```
docker compose --progress plain build
```

Extrait réel de la sortie du build réussi. Seules les lignes finales sont reproduites ; les téléchargements et les étapes intermédiaires sont omis.

```
#16 [mlflow] exporting to image
#16 unpacking to docker.io/library/multimodal-classification-
mlflow:latest 25.2s done
#16 DONE 73.8s

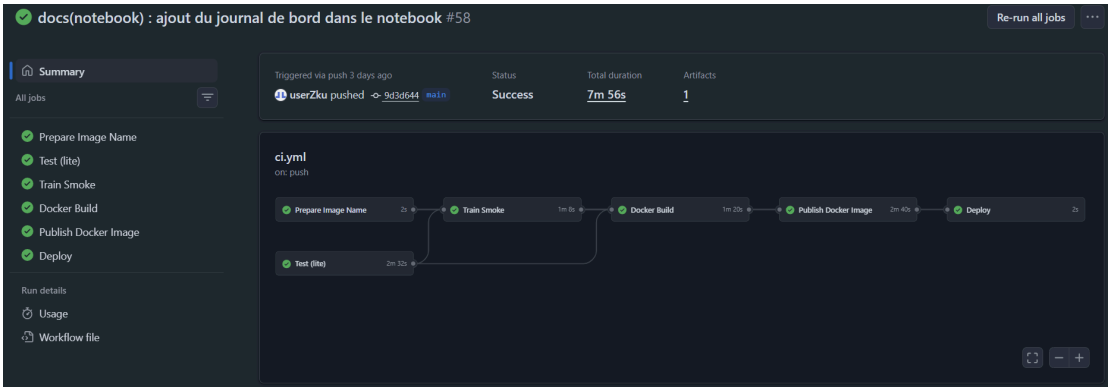
#15 [retrain-cron] exporting to image
#15 unpacking to docker.io/library/multimodal-classification-retrain-
cron:latest 25.2s done
#15 DONE 73.8s

#17 [api] exporting to image
#17 unpacking to docker.io/library/multimodal-classification-api:latest
25.2s done
#17 DONE 73.8s
```

```
#19 [api] resolving provenance for metadata file
#19 DONE 0.1s
#18 [mlflow] resolving provenance for metadata file
#18 DONE 0.1s
#20 [retrain-cron] resolving provenance for metadata file
#20 DONE 0.0s
 Image multimodal-classification-mlflow Built
 Image multimodal-classification-retrain-cron Built
 Image multimodal-classification-api Built
```

Les trois images sont construites et exportées : **API**, **MLflow** et **retrain-cron**. Un build réussi prouve la construction des images, pas à lui seul leur bon fonctionnement en exécution. Les captures de prédiction, Swagger et supervision de la section 8.9 constituent des preuves complémentaires à leurs dates de prise de vue.

Workflow GitHub Actions réussi



La capture montre le **run n°58**, associé au commit `9d3d644` sur la branche `main`, avec le statut **Success** et une durée totale de **7 min 56 s**. Les jobs visibles couvrent la préparation du nom d'image, les tests, l'entraînement de vérification, le build Docker, la publication de l'image et le job de déploiement.

Cette capture complète la section 8.9. Elle porte sur le commit indiqué, pas nécessairement sur la dernière version du projet. Un job `Deploy` vert ne prouve pas à lui seul un déploiement distant : le workflow peut ignorer cette étape si aucun webhook n'est configuré.

Historique des exécutions : GitHub Actions du projet.

---

9. Suivi en production et amélioration continue

Cette section définit le pilotage du modèle après mise en service : surveillance, détection de dérive, boucle de retour et règles de réentraînement. L'objectif est d'assurer la stabilité opérationnelle dans le temps.

9.1 Métriques à surveiller

Les métriques techniques sont instrumentées ; les indicateurs de performance et d'équité ci-dessous constituent un **plan de suivi à mettre en place sur les retours terrain étiquetés**. Les seuls logs d'inférence ne permettent pas de calculer le F1, le rappel ou les erreurs réelles.

Type	Métrique	Seuil ou signal à examiner	Source	Action corrective
Performance modèle	F1 macro hebdomadaire	< baseline - 0,05, en précisant l'effectif étiqueté	Jointure logs et labels terrain	Audit des erreurs et réentraînement
Risque métier	Recall classe 2	< 0,50, avec IC95 et nombre réel de classe 2	Prédictions jointes aux labels terrain	Analyse de sous-détection et recalibration hors test
Équité : détection	Effectif étiqueté, nombre de classe 2 et recall par groupe	Baisse ou écart persistant ; publier les dénominateurs et IC95, sans verdict automatique sur un petit groupe	Espace d'audit autorisé : groupes sources et prédictions étiquetées	Priorité aux <30 ans et aux âges manquants ; rechercher les causes avant correction
Équité : sécurité du filet	Nombre de `2 -> 0`, non escaladés et couverture par groupe	Toute erreur critique non escaladée à examiner ; comparer des périodes de taille suffisante	Labels, classes prédites et décision d'escalade de la version concernée	Revue métier des erreurs résiduelles ; pas de seuil ajusté par nationalité
Équité : charge humaine	Taux d'escalade par groupe et délais de relecture	Écart durable de charge (test : 57,1 % hors UE vs 37,8 % UE) ; capacité validée pour environ 40 % au global	Décisions d'escalade et suivi des relectures ; groupes via jointure séparée autorisée	Vérifier surcharge, délais et couverture réelle des retours
Qualité des données d'audit	Couverture des labels et attributs manquants par groupe	Absence de labels, petits effectifs ou surreprésentation des dossiers escaladés	Base de retours et espace d'audit	Qualifier aussi les dossiers non escaladés pour limiter le biais de sélection
Dérive données	PSI (Population Stability Index)	> 0,20 sur les variables clés	Batch d'analyse à mettre en place	Contrôle des entrées et du préprocessing
Qualité système	Latence p95 API	> 500 ms (cible < 300 ms)	Prometheus	Optimisation des ressources
Disponibilité	Taux d'erreurs HTTP 5xx	> 0,1 %	Prometheus	Investigation système

Référence descriptive de l'audit 7.2.2 : **500 dossiers, dont 90 classe 2** ; **56 hors UE (20 classe 2), 444 UE (70)** ; tranches **<30 : 130 (17), 30–49 : 190 (11), ≥50 : 158 (57)** ; **22**

âges manquants (5). Les taux ne sont pas des objectifs d'équité garantis. Consigner la version du modèle et de la politique, les effectifs, les intervalles et le périmètre de collecte pour chaque comparaison. La collecte de nationalité pour l'audit doit être autorisée et séparée de `/predict` ; aucun ajout au contrat d'inférence n'est réalisé ici.

9.2 Boucle de rétroaction (Feedback Loop)

L'amélioration continue du modèle repose sur le retour du terrain, implémenté de bout en bout (API + stockage + trigger + réentraînement, cf. `docs/FEEDBACK_LOOP.md`) :

1. **Qualification effective** : le délai réel de retour à l'emploi est renseigné par le conseiller, rattaché au `request_id` journalisé lors de la prédiction initiale.
2. **Collecte** : l'endpoint `POST /feedback` valide que le `request_id` correspond bien à une prédiction déjà loguée (404 sinon), rejette les labels invalides (422) et détecte les annotations contradictoires sur un même dossier (409, arbitrage humain) — un renvoi identique reste idempotent (201).
3. **Stockage** : les annotations sont persistées dans `data/feedbacks.db` (SQLite), avec un indicateur `used_for_training` distinguant les feedbacks déjà exploités des nouveaux, afin d'éviter un réentraînement en boucle sur les mêmes données.
4. **Cas signalés** : les dossiers marqués `a_revoir` restent une source prioritaire d'annotation, sans exclure les autres retours conseillers qui alimentent la même table.

9.3 Plan de réentraînement

- **Cadence** : job planifié (`infra/cron/retrain.crontab` , toutes les 6h), sous garde-seuil — il ne s'exécute que si au moins 200 feedbacks non consommés sont disponibles (sinon il sort en succès sans rien faire).
- **Déclencheurs complémentaires** :
  - déclenchement manuel via l'endpoint API `POST /retrain` , le bouton d'action de l'UI, ou `workflow_dispatch` sur `.github/workflows/retrain.yml` ;
  - alerte sur dérive de performance (baisse du F1 macro > 0.05) ou dérive de distribution (PSI > 0.20), à instruire manuellement pour l'instant (cf. section 9.1).
- **Protocole de promotion** (`scripts/promotion.py::decide_promotion` , politique écrite et testée dans `docs/runbooks/decision-log.md` , DEC-028) :
  1. Entraînement du nouveau candidat sur l'historique enrichi des feedbacks joints à leurs features via les logs de `/predict` (scénario S2, même pipeline que la section 5.5).
  2. Évaluation du candidat **et** du modèle en production sur le **même** jeu de test de référence, figé lors du split stratifié de la section 4.1 (`data/reference_set.csv`) — ce jeu n'entre jamais dans l'entraînement.
  3. Décision fondée sur deux métriques, dans cet ordre de priorité : le **recall de la classe 2** ne doit pas régresser de plus de 0,5 point (métrique critique, cf. section 1.4) ; le **F1 macro** peut régresser jusqu'à 1 point, au-delà seulement si le gain de recall classe 2 dépasse 1 point (arbitrage assumé : mieux détecter les risques de longue durée prime sur l'équilibre global).

4. Le candidat remplace le modèle actif dans `models/best_model/` uniquement en cas de promotion ; la décision (promue ou rejetée) est toujours journalisée avec sa raison explicite, et le rejet n'est pas traité comme une erreur.
5. Rechargement dynamique en mémoire par l'API sans interruption de service.

9.4 Robustesse en exploitation

Axe	Métrique opérationnelle	Action en cas de dépassement
Hors-distribution (OOD)	Score de distance / isolation	Journalisation, statut `a_revoir` automatique et escalade conseiller
Drift du seuil de rejet	Taux d'abstention > 25 % pour la règle simple historique ; budget OOF 41 % pour la politique combinée recommandée	Comparer à la baseline de la politique réellement servie avant toute recalibration hors test
Calibration des probabilités	Score de Brier / ECE	Ajustement de la calibration (Isotonic / Sigmoid) sur les prédictions

9.5 Synthèse amélioration continue

Le suivi technique est en place ; le pilotage des performances et de l'équité dépend des labels terrain et d'un cadre de collecte autorisé. L'audit initial révèle des écarts à investiguer, pas une équité acquise. La politique combinée évaluée reste à intégrer à l'API et sa capacité de reprise humaine à valider avant un déploiement élargi.

Annexes

A. Glossaire métier et technique

- **Classe 0 (Rapide)** : retour à l'emploi estimé à moins de 6 mois.
- **Classe 1 (Moyen)** : retour à l'emploi estimé entre 6 et 12 mois.
- **Classe 2 (Risque longue durée)** : risque de chômage prolongé supérieur à 12 mois. Classe critique métier.
- **Erreur critique (2 -> 0)** : usager relevant réellement de la classe 2 prédit à tort en classe 0. Risque le plus préjudiciable.
- **F1 macro** : moyenne non pondérée des F1-scores des 3 classes, adaptée aux cibles déséquilibrées.
- **ROC-AUC OVR macro** : mesure One-vs-Rest de la qualité du classement des probabilités prédites sur les 3 classes.
- **Scénario S2** : pipeline multimodal de production (numérique + diplôme ordinal + catégories nominales + TF-IDF), excluant `nationalite_hors_ue`.
- **Escalade / HITL (Human-in-the-Loop)** : mécanisme de bascule vers une revue humaine lorsque le score de confiance est inférieur au seuil `0,55`.

B. Sources et documentations de référence

- **Dataset** : sujet d'examen CISIA — Promo Upskilling Atlas (mai-octobre 2026), 2 500 enregistrements anonymisés.
- **Frameworks ML** : scikit-learn (pipelines, ColumnTransformer, OrdinalEncoder, OneHotEncoder, TfidfVectorizer), XGBoost, LightGBM.
- **Serviabilité & MLOps** : FastAPI (serving REST), MLflow 3.x (tracking server), Docker Compose, Prometheus & Grafana (supervision).

C. Traçabilité des décisions clés

L'ensemble des arbitrages techniques, éthiques et d'architecture est archivé dans le registre de décisions du projet (`docs/runbooks/decision-log.md`).

Principales décisions de référence :

- `DEC-006` / `DEC-023` : exclusion de `nationalite_hors_ue` en production (scénario S2) pour prévenir toute discrimination directe.
- `DEC-019` / `DEC-023` : encodage ordinal explicite de `niveau_diplome` (`Sans diplôme < Bac < Bac+2 < Bac+5`).
- `DEC-017` : politique d'abstention basée sur un seuil de confiance (`0,55`) et revue humaine.
- `DEC-020` / `DEC-024` : API FastAPI, observabilité Prometheus/Grafana et serveur MLflow conteneurisé dans Docker Compose.
- `DEC-026` : dashboard Grafana étendu avec les métriques de sortie du modèle (classes prédites, confiance, abstention).
- `DEC-027` : boucle de feedback (endpoint `/feedback` , stockage SQLite, trigger cron sur seuil de feedbacks non consommés).
- `DEC-028` : politique de promotion testable, recall classe 2 prioritaire sur le F1 macro (cf. section 9.3).

D. Journal de bord complet

Le journal détaillé est conservé dans reports/journal/journal-de-bord.ipynb. La version ci-dessous en reprend toutes les sessions sous une forme plus courte, adaptée au notebook de certification.

E. Journal de bord synthétisé

Entrée 1 — 16/07/2026 : initialisation du projet

- **Actions** : lecture du brief, contrôle du dataset, création de l'arborescence et des premiers livrables.
- **Résultats** : dépôt organisé autour du notebook, du code, des données, des tests et de la soutenance.
- **Décision** : adopter une structure de projet complète pour assurer la traçabilité.
- **Vigilance** : éviter que les nombreux artefacts techniques éloignent du livrable officiel.

Entrée 2 — 16/07/2026 : cadrage métier

- **Actions** : formalisation du problème multiclasse, des bénéficiaires, des critères de succès et des risques liés à la nationalité.
- **Résultats** : erreurs `2 -> 0` identifiées comme risque métier prioritaire ; scénarios S1/S2 définis.
- **Décision** : comparer explicitement performance, conformité et biais.
- **Vigilance** : documenter toute utilisation d'une variable sensible ou d'un proxy.

Entrée 3 — 16/07/2026 : données et EDA

- **Actions** : documentation des sources, des valeurs manquantes, des distributions, de l'équilibre des classes et des signaux de biais.
- **Résultats** : cible modérément déséquilibrée, variables sensibles identifiées et faible diversité du texte constatée.
- **Décision** : relier chaque constat EDA à une décision de préparation ou de gouvernance.
- **Vigilance** : ne pas sur-interpréter un signal texte peu diversifié.

Entrée 4 — 16/07/2026 : préparation des données

- **Actions** : mise en place du split stratifié, du préprocessing multimodal, des quatre scénarios et des assertions anti-fuite.
- **Résultats** : protocole train/test reproductible et scénarios S1 à S4 comparables.
- **Décision** : ajuster les transformations uniquement sur le train.
- **Vigilance** : maintenir l'exclusion de l'identifiant technique et contrôler les colonnes sensibles.

Entrée 5 — 24/07/2026 : refonte du livrable de certification

- **Actions** : relecture du README et du notebook, harmonisation du style et amélioration des transitions.
- **Résultats** : récit plus lisible pour le jury de certification, sans suppression des limites méthodologiques.
- **Décision** : présenter le dépôt comme un livrable répondant aux attentes de la certification.
- **Vigilance** : ne pas remplacer les preuves techniques par des affirmations générales déconnectées des preuves.

Entrée 6 — 24/07/2026 : modélisation et arbitrages initiaux

- **Actions** : comparaison CV des modèles, tuning ciblé, évaluation finale et rédaction de l'arbitrage S1/S2.
- **Résultats** : XGBoost multimodal identifié comme candidat performant ; tension observée entre score global et rappel de classe 2.
- **Décision** : transformer les scores en arbitrage métier plutôt qu'en simple classement technique.
- **Vigilance** : la meilleure performance brute ne suffit pas à justifier la production si elle repose sur une variable sensible.

Entrée 7 — 27/07/2026 : interprétation et escalade

- **Actions** : clarification des erreurs critiques, des seuils de confiance, de l'escalade humaine et de la limite du texte peu diversifié.
- **Résultats** : distinction entre règle simple et politique combinée ; coût de la reprise humaine rendu explicite.
- **Décision** : maintenir un modèle assistif avec revue humaine des cas incertains.
- **Vigilance** : calibrer le seuil sans utiliser le jeu de test pour éviter une évaluation optimiste.

Entrée 8 — 29/07/2026 : API, monitoring et MLflow

- **Actions** : stabilisation de FastAPI, ajout des métriques Prometheus, des logs JSONL, de Grafana, de Docker Compose et du tracking MLflow.
- **Résultats** : chaîne training -> serving -> monitoring démontrable localement.
- **Décision** : conserver une API minimaliste centrée sur `predict`, `retrain`, `health` et `metrics`.
- **Vigilance** : la sécurité d'accès et le backend MLflow restent des sujets de maturité ultérieure.

Entrée 9 — 29/07/2026 : interface de démonstration

- **Actions** : création de l'UI statique, simplification du formulaire et intégration à l'API.
- **Résultats** : parcours de démonstration plus fluide et cohérence renforcée avec le contrat API.
- **Décision** : privilégier une interface simple et fiable plutôt qu'une application front-end surdimensionnée.
- **Vigilance** : `usager_id` reste un identifiant de traçabilité, jamais une feature du modèle.

Entrée 10 — 04/09/2026 : consolidation S2 et industrialisation

- **Actions** : alignement du code, du notebook et de Docker sur S2 ; exclusion de la variable sensible ; diplôme ordinal ; ajout du ROC-AUC et de MLflow conteneurisé.
- **Résultats** : S2 devient cohérent de l'entraînement à l'inférence, avec suivi des proxys et des erreurs critiques.
- **Décision** : rendre S2 seul éligible à la production ; conserver S1 comme référence expérimentale.
- **Vigilance** : les proxys indirects, notamment l'âge et le texte libre, nécessitent encore un audit par sous-population.

Entrée 11 — 07/09/2026 : finalisation des livrables

- **Actions** : refonte de l'IHM, amélioration du notebook, génération des captures, mise à jour de la documentation et préparation de la soutenance.
- **Résultats** : chaîne complète présentable : modèle, API, UI, monitoring, MLflow, Docker et CI/CD.
- **Décision** : faire de la démonstration de bout en bout un axe central de la soutenance.

- **Vigilance** : conserver les limites du modèle et les garde-fous visibles dans l'interface et le discours.

Entrée 12 — 14/09/2026 : feedback et amélioration continue

- **Actions** : ajout du retour terrain dans SQLite, du réentraînement, de la promotion conditionnelle, du rollback, du cron et du dashboard orienté modèle.
- **Résultats** : boucle MLOps complète avec décision de promotion, historique des modèles et réversibilité.
- **Décision** : prioriser le recall de la classe 2 dans la politique de promotion, avec le F1 macro comme garde-fou secondaire.
- **Vigilance** : la jointure des retours terrain avec les variables et les seuils de tolérance devront être réévalués sur des cycles réels.

Entrée 13 - 01/10/2026 : seuil hors test et compromis de reprise humaine

- **Actions** : calibration avec des probabilités OOF sur cinq folds du train ; comparaison des politiques simple et combinée ; correction des chiffres et arbitrages du notebook après les retours de la formatrice.
- **Résultats** : politique combinée à 0,70, couverture OOF 72,5 % et escalade 40,2 % ; sur le test, 200/500 dossiers signalés, 8/10 erreurs critiques signalées et 2 non escaladées.
- **Revirement argumenté** : la règle 0,55 et un budget de 25 % ne suffisent pas ; accepter une reprise proche de 40 % (borne OOF 41 %) pour une cible OOF de 70 % de couverture, à valider avec la capacité réelle de l'agence. Le 80 % sur test est un résultat, pas une cible tenue en OOF.
- **Vigilance** : le seuil est désormais choisi hors test, mais ce test a déjà été consulté historiquement ; une confirmation indépendante reste nécessaire. La politique combinée n'est pas encore intégrée à l'API, qui conserve 0,55.

Entrée 14 - 04/10/2026 : équité mesurée et incertitude du filet humain

- **Actions** : audit du test S2 par nationalité et trois tranches d'âge, avec catégorie séparée pour les âges manquants ; calcul des dénominateurs, rappels, erreurs critiques, erreurs non escaladées et escalades ; mise à jour de 7.3 et 9.1.
- **Résultats** : 56 hors UE (20 classe 2), rappel 45,0 %, 2/2 erreurs critiques escaladées ; 444 UE (70 classe 2), rappel 55,7 %, 6/8 erreurs escaladées. Le rappel des <30 ans est de 11,8 %, contre 73,7 % chez les ≥50 ans : l'exclusion n'a pas supprimé les écarts.
- **Décision** : suivre les écarts et la charge humaine, sans proclamer l'équité ni retoucher le seuil sur le test. Communiquer 200/500 dossiers signalés, 8/10 erreurs critiques signalées et 2 non escaladées ; IC95 de couverture ≈49–94 %, vu le petit dénominateur.
- **Vigilance** : usage distinct de la nationalité uniquement pour l'audit agrégé, à encadrer avec le DPO ; absence d'ajout aux features ou à l'API. Les relectures réellement effectuées et les corrections humaines ne sont pas mesurées. Confirmation externe et collecte de labels non biaisée nécessaires.